

JointDNN: An Efficient Training and Inference Engine for Intelligent Mobile Cloud Computing Services

Amir Erfan Eshratifar, Mohammad Saeed Abrishami, and Massoud Pedram, *Fellow, IEEE*

Abstract—Deep learning models are being deployed in many mobile intelligent applications. End-side services, such as intelligent personal assistants, autonomous cars, and smart home services often employ either simple local models on the mobile or complex remote models on the cloud. However, recent studies have shown that partitioning the DNN computations between the mobile and cloud can increase the latency and energy efficiencies. In this paper, we propose an efficient, adaptive, and practical engine, JointDNN, for collaborative computation between a mobile device and cloud for DNNs in both inference and training phase. JointDNN not only provides an energy and performance efficient method of querying DNNs for the mobile side but also benefits the cloud server by reducing the amount of its workload and communications compared to the cloud-only approach. Given the DNN architecture, we investigate the efficiency of processing some layers on the mobile device and some layers on the cloud server. We provide optimization formulations at layer granularity for forward- and backward-propagations in DNNs, which can adapt to mobile battery limitations and cloud server load constraints and quality of service. JointDNN achieves up to 18 and 32 times reductions on the latency and mobile energy consumption of querying DNNs compared to the status-quo approaches, respectively.

Index Terms—deep neural networks, intelligent services, mobile computing, cloud computing

1 INTRODUCTION

DNN architectures are promising solutions in achieving remarkable results in a wide range of machine learning applications, including, but not limited to computer vision, speech recognition, language modeling, and autonomous cars. Currently, there is a major growing trend in introducing more advanced DNN architectures and employing them in end-user applications. The considerable improvements in DNNs are usually achieved by increasing computational complexity which requires more resources for both training and inference [1]. Recent research directions to make this progress sustainable are: development of Graphical Processing Units (GPUs) as the vital hardware component of both servers and mobile devices [2], design of efficient algorithms for large-scale distributed training [3] and efficient inference [4], compression and approximation of models [5], and most recently introducing collaborative computation of cloud and fog as known as dew computing [6].

Deployment of cloud servers for computation and storage is becoming extensively favorable due to technical advancements and improved accessibility. Scalability, low cost, and satisfactory Quality of Service (QoS) made offloading to cloud a typical choice for computing-intensive tasks. On the other side, mobile-device are being equipped with more powerful general-purpose CPUs and GPUs. Very recently there is a new trend in hardware companies to design dedicated chips to better tackle machine-learning tasks. For example, Apple's A11 Bionic chip [7] used in iPhone X uses a neural engine in its GPU to speed up the DNN queries of applications such as face identification and facial motion capture [8].

In the status-quo approaches, there are two methods for DNN inference: mobile-only and cloud-only. In simple models, a mobile device is sufficient for performing all the computations. In the case of complex models, the raw input data (image, video stream, voice, etc.) is uploaded to and then the required computations are performed on the cloud server. The results of the task are later downloaded to the device. The effects of raw input and feature compression are studied in [9] and [10].

Despite the recent improvements of the mobile devices mentioned earlier, the computational power of mobile devices is still significantly weaker than the cloud ones. Therefore, the mobile-only approach can cause large inference latency and failure in meeting QoS. Moreover, embedded devices undergo major energy consumption constraints due to battery limits. On the other hand, cloud-only suffers communication overhead for uploading the raw data and downloading the outputs. Moreover, slowdowns caused by service congestion, subscription costs, and network dependency should be considered as downsides of this approach [11].

The superiority and persistent improvement of DNNs depend heavily on providing a huge amount of training data. Typically, this data is collected from different resources and later fed into a network for training. The final model can then be delivered to different devices for inference functions. However, there is a trend of applications requiring adaptive learning in online environments, such as self-driving cars and security drones [12] [13]. Model parameters in these smart devices are constantly being changed based on their continuous interaction with their environment. The complexity of these architectures with an increasing number of parameters and current cloud-only methods for DNN training implies a constant communication cost and the burden of increased energy consumption for mobile devices.

• All authors are with the Department of Electrical Engineering, University of Southern California, Los Angeles, CA 90089-2562 USA (eshratif@usc.edu, abri442@usc.edu, pedram@usc.edu).

JointDNN: 智能移动云计算服务的高效训练和推理引擎

Amir Erfan Eshratifar、Mohammad Saeed Abrishami 和 Massoud Pedram, *Fellow, IEEE*

摘要: 深度学习模型正在许多移动智能应用中部署。终端服务, 例如智能个人助理、自动驾驶汽车和智能家居服务, 通常采用移动设备上的简单本地模型或云上的复杂远程模型。然而, 最近的研究表明, 在移动设备和云端之间划分 DNN 计算可以增加延迟和能源效率。在本文中, 我们提出了一种高效、自适应且实用的引擎 JointDNN, 用于移动设备和云之间的 DNN 推理和训练阶段的协作计算。JointDNN 不仅为移动端提供了一种能源和性能高效的 DNN 查询方法, 而且与纯云方法相比, 通过减少其工作负载和通信量, 也使云服务器受益。给定 DNN 架构, 我们研究了移动设备上某些层和云服务器上某些层的处理效率。我们为 DNN 中的前向和后向传播提供层粒度的优化公式, 可以适应移动电池限制、云服务器负载限制和服务质量。与现有方法相比, JointDNN 分别将查询 DNN 的延迟和移动能耗降低了 18 倍和 32 倍。

索引术语——深度神经网络、智能服务、移动计算、云计算

1 简介

DNN 架构是有前途的解决方案, 可在广泛的机器学习应用中取得显着的成果, 包括但不限于计算机视觉、语音识别、语言建模和自动驾驶汽车。目前, 引入更先进的 DNN 架构并将其应用于最终用户应用程序是一个主要的增长趋势。DNN 的显着改进通常是通过增加计算复杂性来实现的, 这需要更多的资源用于训练和推理 [1]。使这一进展可持续的最新研究方向是: 开发图形处理单元 (GPU) 作为服务器和移动设备的重要硬件组件 [2], 设计用于大规模分布式训练 [3] 和高效推理 [4] 的有效算法, 模型的压缩和近似 [5], 以及最近引入的云和雾协同计算 (称为露水计算) [6]。

由于技术进步和可访问性的提高, 用于计算和存储的云服务器的部署变得越来越受欢迎。可扩展性、低成本和令人满意的服务质量 (QoS) 使得云计算成为计算密集型任务的典型选择。另一方面, 移动设备正在配备更强大的通用 CPU 和 GPU。最近, 硬件公司出现了一种新趋势, 即设计专用芯片以更好地处理机器学习任务。例如, iPhone X 中使用的苹果 A11 仿生芯片 [7] 在其 GPU 中使用神经引擎来加速面部识别和面部动作捕捉等应用程序的 DNN 查询 [8]。

在现有的方法中, 有两种 DNN 推理方法: 仅移动设备和仅云。在简单模型中, 移动设备足以执行所有计算。对于复杂模型, 原始输入数据 (图像、视频流、语音等) 被上传到云服务器, 然后在云服务器上执行所需的计算。任务结果稍后会下载到设备。[9] 和 [10] 研究了原始输入和特征压缩的影响。

尽管前面提到的移动设备最近有所改进, 但移动设备的计算能力仍然明显弱于云设备。因此, 仅移动设备的方法可能会导致较大的推理延迟并无法满足 QoS。此外, 由于电池限制, 嵌入式设备面临着重大的能耗限制。另一方面, 仅云会遭受上传原始数据和下载输出的通信开销。此外, 由服务拥塞、订阅成本和网络依赖性引起的速度下降应该被视为这种方法的缺点 [11]。

DNN 的优越性和持续改进在很大程度上取决于提供大量的训练数据。通常, 这些数据是从不同的资源收集的, 然后输入网络进行训练。然后可以将最终模型传送到不同的设备以实现推理功能。然而, 在线环境中存在需要自适应学习的应用趋势, 例如自动驾驶汽车和安全无人机 [12][13]。这些智能设备中的模型参数会根据其与环境持续交互而不断变化。这些架构的复杂性以及越来越多的参数和当前用于 DNN 训练的仅云方法意味着恒定的通信成本以及移动设备能耗增加的负担。

• All authors are with the Department of Electrical Engineering, University of Southern California, Los Angeles, CA 90089-2562 USA (eshratif@usc.edu, abri442@usc.edu, pedram@usc.edu).

The main difference of collaborative training and cloud-only training is that the data transferred in the cloud-only approach is the input data and model parameters but in the collaborative approach, it is layer(s)'s output and a portion of model parameters. Therefore, the amount of data communicated can be potentially decreased [14].

Automatic partitioning of computationally extensive tasks over the cloud for optimization of performance and energy consumption has been already well-studied [15]. Most recently, scalable distributed hierarchy structures between the end-user device, edge, and cloud have been suggested [16] which are specialized for DNN applications. However, exploiting the layer granularity of DNN architectures for run-time partitioning has not been studied thoroughly yet.

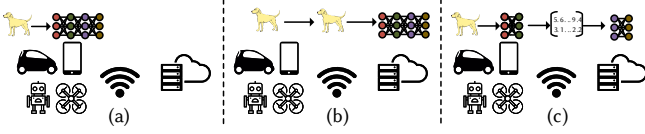


Fig. 1: Different computation partitioning methods. (a) Mobile only: computation is completely done on the mobile device. (b) Cloud only: raw input data is sent to the cloud server, computations are done on the cloud server and results are sent back to the mobile device. (c) JointDNN: DNN architecture is partitioned at the granularity of layers, each layer can be computed either on cloud or mobile.

In this work, we are investigating the inference and training of DNNs in a **joint** platform of mobile and cloud as an alternative to the current single-platform methods as illustrated in Figure 1. Considering DNN architectures as an ordered sequence of layers, and the possibility of computation of every layer either on mobile or cloud, we can model the DNN structure as a Directed Acyclic Graph (DAG). The parameters of our real-time adaptive model are dependent on the following factors: mobile/cloud hardware and software resources, battery capacity, network specifications, and QoS. Based on this modeling, we show that the problem of finding the optimal computation schedule for different scenarios, i.e. best performance or energy consumption, can be reduced to the polynomial-time shortest path problem.

To present realistic results, we made experiments with fair representative hardware of mobile device and cloud. To model the communication costs between platforms, we used various mobile network technologies and the most recent reports on their specifications in the U.S.

DNN architectures can be categorized based on functionality. These differences enforce specific type and order of layers in architecture, directly affecting the partitioning result in the collaborative method. For discriminative models, used in recognition applications, the layer size gradually decreases going from input toward output as shown in Figure 2. This sequence suggests the computation of the first few layers on the mobile device to avoid excessive communication cost of uploading large raw input data. On the other hand, the growth of the layer output size from input to output in generative models which are used for synthesizing new data, implies the possibility of uploading a small input vector to the cloud and later downloading one of the last layers and performing the rest of computations on the mobile device for better efficiency. Interesting mobile ap-

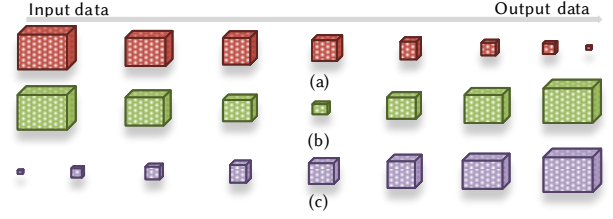


Fig. 2: Typical layer size in (a) Discriminative (b) Autoencoder (c) Generative models.

plications like image-to-image translation are implemented with autoencoder architectures whose middle layers sizes are smaller compared to their input and output. Consequently, to avoid huge communication costs, we expect the first and last layers to be computed on the mobile device in our collaborative approach. We examined eight well-known DNN benchmarks selected from these categories to illustrate their differences in the collaborative computation approach.

As we will see in Section 4, the communication between the mobile and cloud is the main bottleneck for both performance and energy in the collaborative approach. We investigated the specific characteristics of CNN layer outputs and introduced a loss-less compression approach to reduce the communication costs while preserving the model accuracy.

State-of-the-art work for collaborative computation of DNNs [14] only considers one offloading point, assigning computation of its previous layers and next layers on the mobile and cloud platforms, respectively. We show that this approach is non-generic and fails to be optimal, and introduced a new method granting the possibility of computation on either platform for each layer independent of other layers. Our evaluations show that JointDNN significantly improves the latency and energy up to $3\times$ and $7\times$ respectively compared to the status-quo single platform approaches without any compression. The main contributions of this paper can be listed as:

- Introducing a new approach for the collaborative computation of DNNs between the mobile and cloud
- Formulating the problem of optimal computation scheduling of DNNs at layer granularity in the mobile cloud computing environment as the shortest path problem and integer linear programming (ILP)
- Examining the effect of compression on the outputs of DNN layers to improve communication costs
- Demonstrating the significant improvements in performance, mobile energy consumption, and cloud workload achieved by using **JointDNN**

2 PROBLEM DEFINITION AND MODELING

In this section, we explain the general architecture of DNN layers and our profiling method. Moreover, we elaborate on how cost optimization can be reduced to the shortest path problem by introducing the JointDNN graph model. Finally, we show how the constrained problem is formulated by setting up ILP.

2.1 Energy and Latency Profiling

There are three methods in measuring the latency and energy consumption of each layer in neural networks [17]:

协作训练和纯云训练的主要区别在于，纯云方法中传输的数据是输入数据和模型参数，而在协作方法中，它是层的输出和部分模型参数。因此，传输的数据量可能会减少[14]。

通过云自动划分计算量大的任务以优化性能和能耗已经得到了充分研究[15]。最近，有人提出了最终用户设备、边缘和云之间的可扩展分布式层次结构[16]，专门用于 DNN 应用。然而，利用 DNN 架构的层粒度进行运行时划分尚未得到彻底研究。

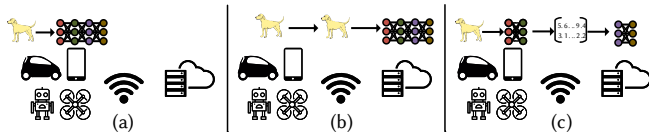


图 1：不同的计算划分方法。(a) 仅移动设备：计算完全在移动设备上完成。(b) 仅云：原始输入数据被发送到云服务器，计算在云服务器上完成，结果被发送回移动设备。(c) Joint DNN：DNN 架构按层的粒度进行划分，每层都可以在云端或移动设备上计算。

在这项工作中，我们正在研究移动和云联合平台中 DNN 的推理和训练，作为当前单平台方法的替代方案，如图 1 所示。将 DNN 架构视为层的有序序列，以及在移动或云上计算每一层的可能性，我们可以将 DNN 结构建模为有向无环图（DAG）。我们的实时自适应模型的参数取决于以下因素：移动/云硬件和软件资源、电池容量、网络规格和服务质量。基于此建模，我们表明，为不同场景（即最佳性能或能耗）找到最佳计算调度的问题可以简化为多项式时间最短路径问题。

为了呈现真实的结果，我们使用移动设备和云的具有代表性的硬件进行了实验。为了对平台之间的通信成本进行建模，我们使用了各种移动网络技术以及美国有关其规范的最新报告。

DNN 架构可以根据功能进行分类。这些差异强制执行架构中层的特定类型和顺序，直接影响协作方法中的划分结果。对于识别应用中使用的判别模型，层大小从输入到输出逐渐减小，如图 2 所示。该序列建议在移动设备上计算前几层，以避免上传大量原始输入数据的过多通信成本。另一方面，在用于合成新数据的生成模型中，从输入到输出的层输出大小的增长意味着可以将小输入向量上传到云端，然后下载最后一层并在移动设备上执行其余计算，以提高效率。有趣的移动应用程序 -

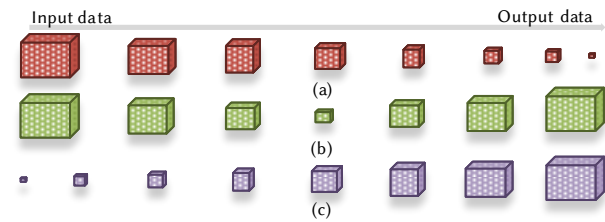


图 2：(a) 判别式 (b) 自动编码器 (c) 生成模型中的典型层大小。

像图像到图像翻译这样的应用是通过自动编码器架构实现的，其中间层的大小与其输入和输出相比更小。因此，为了避免巨大的通信成本，我们希望在我们的协作方法中第一层和最后一层在移动设备上计算。我们检查了从这些类别中选择的八个著名的 DNN 基准，以说明它们在协作计算方法中的差异。

正如我们将在第 4 节中看到的，移动设备和云之间的通信是协作方法中性能和能量的主要瓶颈。我们研究了 CNN 层输出的具体特征，并引入了无损压缩方法来降低通信成本，同时保持模型的准确性。

DNN 协作计算的最先进工作 [14] 仅考虑一个卸载点，分别在移动平台和云平台上分配其前一层和下一层的计算。我们证明这种方法是非通用的并且不是最优的，并且引入了一种新方法，允许在任一平台上为独立于其他层的每一层进行计算。我们的评估表明，与没有任何压缩的现状单平台方法相比，JointDNN 将延迟和能量分别显著提高了 3× 和 7×。本文的主要贡献可归纳为：

- 引入一种移动和云之间 DNN 协作计算的新方法
- 将移动云计算环境中层粒度的 DNN 最优计算调度问题表述为最短路径问题和整数线性规划（ILP）
- 检查压缩对 DNN 层输出的影响以提高通信成本
- 展示通过使用 JointDNN 在性能、移动能耗和云工作负载方面实现的显著改进

2 问题定义和建模

在本节中，我们将解释 DNN 层的一般架构和我们的分析方法。此外，我们详细阐述了如何通过引入 JointDNN 图模型将成本优化简化为最短路径问题。最后，我们展示了如何通过设置 ILP 来表述约束问题。

2.1 能量和延迟分析

测量神经网络中每层的延迟和能耗有三种方法[17]：

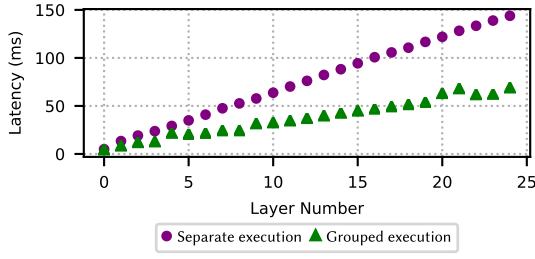


Fig. 3: Latency of grouped and separated execution of convolution operator.

Statistical Modeling: In this method, a regression model over the configurable parameters of operators (e.g. filter size in the convolution) can be used to estimate the associated latency and energy. This method is prone to large errors because of the inter-layer optimizations performed by DNN software packages. Therefore, it is necessary to consider the execution of several consecutive operators grouped during profiling. Many of these software packages are proprietary, making access to inter-layer optimization techniques impossible.

In order to illustrate this issue, we designed two experiments with 25 consecutive convolutions on NVIDIA Pascal™ GPU using cuDNN® library [18]. In the first experiment, we measure the latency of each convolution operator separately and set the total latency as the sum of them. In the second experiment, we execute the grouped convolutions in a single kernel together and measure the total latency. All parameters are located on the GPU's memory in both experiments, avoiding any data transfer from the main memory to make sure results are exactly representing the actual computation latency.

As we see in Figure 3, there is a large error gap between separated and grouped execution experiments which grows as the number of convolutions is increased. This observation confirms that we need to profile grouped operators to have more accurate estimations. Considering the various consecutive combination of operators and different input sizes, this method requires a very large number of measurements, not to mention the need for a complex regression model.

Analytical Modeling: To derive analytical formulations for estimating the latency and energy consumption, it is required to obtain the exact hardware and software specifications. However, the state-of-the-art in latency modeling of DNNs [19] fails to estimate layer-level delay within an acceptable error bound, for instance, underestimating the latency of a fully connected layer with 4096 neurons by around 900%. Industrial developers do not reveal the detailed hardware architecture specifications and the proprietary parallel computing architectures such as CUDA®, therefore, the analytical approach could be quite challenging [20].

Application-specific Profiling: In this method, the DNN architecture of the application being used is profiled in run-time. The number of applications in a mobile device using neural networks is generally limited. In conclusion, this method is more feasible, promising higher accuracy estimations. We have chosen this method for the estimation of energies and latencies in the experiments of this paper.

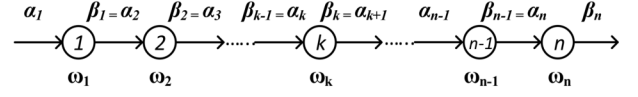


Fig. 4: Computation model in linear topology.

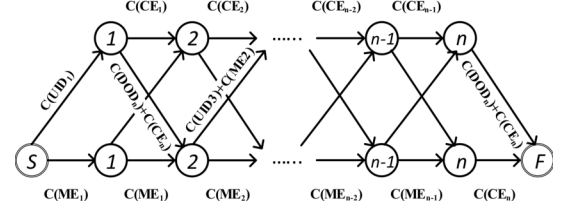


Fig. 5: Graph representation of mobile cloud computing optimal scheduling problem for linear topology.

2.2 JointDNN Graph Model

First, we assume that a DNN is presented by a sequence of distinct layers with a linear topology as depicted in Figure 4. Layers are executed sequentially, with output data generated by one layer feeds into the input of the next one. We denote the input and output data sizes of k^{th} layer as α_k and β_k , respectively. Denoting the latency (energy) of layer k as ω_k , where $k = 1, 2, \dots, n$, the total latency (energy) of querying the DNN is $\sum_{k=1}^n \omega_k$.

The mobile cloud computing optimal scheduling problem can be reduced to the shortest path problem, from node S to F , in the graph of Figure 5. **Mobile Execution** cost of the k^{th} layer ($C(ME_k)$) is the cost of executing the k^{th} layer in the mobile while the cloud server is idle. **Cloud Execution** cost of the k^{th} layer ($C(CE_k)$) is the executing cost of the k^{th} layer in the cloud server while the mobile is idle. **Uploading the Input Data** cost of the k^{th} layer is the cost of uploading output data of the $(k-1)^{th}$ layer to the cloud server (UID_k). **Downloading the Input Data** cost of the k^{th} layer is the cost of downloading output data of the $(k-1)^{th}$ layer to the mobile (DOD_k). The costs can refer to either latency or energy. However, as we showed in Section 2.1, the assumption of linear topology in DNNs is not true and we need to consider all the consecutive grouping of the layers in the network. This fact suggests the replacement of linear topology by a tournament graph as depicted in Figure 6. We define the parameters of this new graph, *JointDNN graph model*, in Table 1.

In this graph, node $C_{i,j}$ represents that the layers i to j are computed on the cloud server, while node $M_{i,j}$ represents that the layers i to j are computed on the mobile device. An edge between two adjacent nodes in JointDNN graph model is associated with four possible cases: 1) A transition from the mobile to the mobile, which

TABLE 1: Parameter Definition of Graph Model

Param.	Description of Cost
$CE_{i,j}$	Executing layers i to j on the cloud
$ME_{i,j}$	Executing layers i to j on the mobile
$ED_{i,j}$	$CE_{i,j} + DOD_j$
$EU_{i,j}$	$ME_{i,j} + UID_j$
ϕ_k	All the following edges: $\forall i = 1 : k-1$ $ED_{i,k-1}$
Ω_k	All the following edges: $\forall i = 1 : k-1$ $ME_{i,k-1}$
Ψ_k	All the following edges: $\forall i = 1 : k-1$ $EU_{i,k-1}$
Γ_k	All the following edges: $\forall i = 1 : k-1$ $CE_{i,k-1}$
Π_m	All the following edges: $\forall i = 1 : n$ $ME_{i,n}$
Π_c	All the following edges: $\forall i = 1 : n$ $ED_{i,n}$
U_1	Uploading the input of the first layer

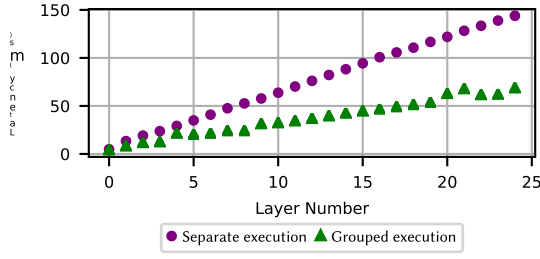


图 3: 卷积算子的分组和分离执行的延迟。

统计建模：在此方法中，可使用算子可配置参数（例如卷积中的滤波器大小）的回归模型来估计相关的延迟和能量。由于 DNN 软件包执行层间优化，该方法容易出现较大误差。因此，有必要在分析过程中考虑分组的多个连续运算符的执行。其中许多软件包都是专有的，因此无法使用层间优化技术。

为了说明这个问题，我们使用 cuDNN[®] 库 [18] 在 NVIDIA Pascal[™] GPU 上设计了两个具有 25 个连续卷积的实验。在第一个实验中，我们分别测量每个卷积算子的延迟，并将总延迟设置为它们的总和。在第二个实验中，我们在单个内核中一起执行分组卷积并测量总延迟。在两个实验中，所有参数都位于 GPU 的内存中，避免了来自主内存的任何数据传输，以确保结果准确代表实际的计算延迟。

如图 3 所示，分离执行实验和分组执行实验之间存在很大的误差差距，该误差差距随着卷积数量的增加而增大。这一观察结果证实，我们需要对分组的运算符进行分析才能获得更准确的估计。考虑到算子的各种连续组合和不同的输入大小，该方法需要大量的测量，更不用说需要复杂的回归模型。

分析建模：为了导出用于估计延迟和能耗的分析公式，需要获得准确的硬件和软件规格。然而，最先进的 DNN 延迟建模 [19] 无法在可接受的误差范围内估计层级延迟，例如，将具有 4096 个神经元的全连接层的延迟低估了约 900%。工业开发人员不会透露详细的硬件架构规范和专有的并行计算架构（例如 CUDA[®]），因此，分析方法可能相当具有挑战性 [20]。

应用程序特定的分析：在此方法中，正在使用的应用程序的 DNN 架构在运行时进行分析。使用神经网络的移动设备中的应用程序数量通常是有限的。综上所述，该方法更加可行，估计精度更高。我们在本文的实验中选择了这种方法来估计能量和延迟。

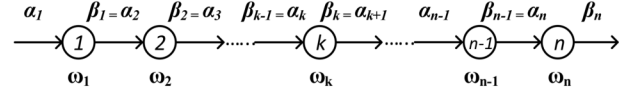


Fig. 4: Computation model in linear topology.

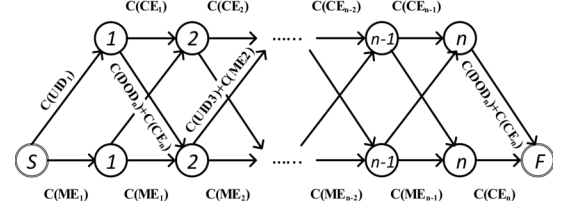


图 5: 线性拓扑移动云计算最优调度问题的图形表示。

2.2 JointDNN图模型

首先，我们假设 DNN 由一系列具有线性拓扑的不同层组成，如图 4 所示。各层按顺序执行，一层生成的输出数据输入到下一层的输入中。我们将 k^{th} 层的输入和输出数据大小分别表示为 α_k 和 β_k 。将第 k 层的延迟（能量）表示为 ω_k ，其中 $k = 1, 2, \dots, n$ ，查询 DNN 的总延迟（能量）为 $\sum_{k=1}^n \omega_k$ 。

移动云计算最优调度问题可以简化为从节点 S 到 F 的最短路径问题，如图 5 所示。 k^{th} 层 ($C(ME_k)$) 的移动执行成本是云服务器空闲时在移动设备中执行 k^{th} 层的成本。 k^{th} 层 ($C(CE_k)$) 的云执行成本是移动设备空闲时云服务器中 k^{th} 层的执行成本。上传 k^{th} 层的输入数据成本是将 $(k-1)^{th}$ 层的输出数据上传到云服务器 (UID_k) 的成本。下载 k^{th} 层的输入数据成本是将 $(k-1)^{th}$ 层的输出数据下载到移动设备 (DOD_k) 的成本。成本可以指延迟或能源。然而，正如我们在 2.1 节中所示，DNN 中线性拓扑的假设并不正确，我们需要考虑网络中层的所有连续分组。这一事实建议用锦标赛图代替线性拓扑，如图 6 所示。我们在表 1 中定义了这个新图 JointDNN graph model 的参数。

在此图中，节点 $C_{i,j}$ 表示层 i 至 j 是在云服务器上计算的，而节点 $M_{i,j}$ 表示层 i 至 j 是在移动设备上计算的。JointDNN 图模型中两个相邻节点之间的边与四种可能的情况相关联：1) 从移动设备到移动设备的转换，其中

表1: 图模型参数定义

Param.	Description of Cost
$CE_{i,j}$	Executing layers i to j on the cloud
$ME_{i,j}$	Executing layers i to j on the mobile
$ED_{i,j}$	$CE_{i,j} + DOD_j$
$EU_{i,j}$	$ME_{i,j} + UID_j$
ϕ_k	All the following edges: $\forall i = 1 : k-1 \ ED_{i,k-1}$
Ω_k	All the following edges: $\forall i = 1 : k-1 \ ME_{i,k-1}$
Ψ_k	All the following edges: $\forall i = 1 : k-1 \ EU_{i,k-1}$
Γ_k	All the following edges: $\forall i = 1 : k-1 \ CE_{i,k-1}$
Π_m	All the following edges: $\forall i = 1 : n \ ME_{i,n}$
Π_c	All the following edges: $\forall i = 1 : n \ ED_{i,n}$
U_1	Uploading the input of the first layer

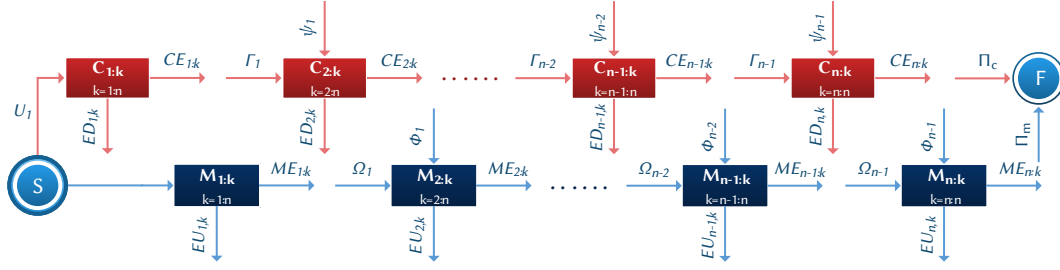


Fig. 6: JointDNN graph model. The shortest path from S to F determines the schedule of executing the layers on mobile or cloud.

only includes the mobile computation cost ($ME_{i,j}$) 2) A transition from the cloud to the cloud, which only includes the cloud computation cost ($CE_{i,j}$) 3) A transition from the mobile to the cloud, which includes the mobile computation cost and uploading cost of the inputs of the next node ($EU_{i,j} = ME_{i,j} + UID_{j+1}$) 4) A transition from the cloud to the mobile, which includes the cloud computation cost and downloading cost of the inputs of the next node ($ED_{i,j} = CE_{i,j} + DOD_{j+1}$). Under this formulation, we can transform the computation scheduling problem to finding the shortest path from S to F . Residual networks are a class of powerful and easy-to-train architectures of DNNs [21]. In residual networks, as depicted in Figure 7 (a), the output of one layer is fed into another layer with a distance of at least two. Thus, we need to keep track of the source layer (node 2 in Figure 7) to know that this layer is computed on the mobile or the cloud. Our standard graph model has a memory of one which is the very previous layer. We provide a method to transform the computation graph of this type of network to our standard model, JointDNN graph.

In this regard, we add two additional chains of size $k-1$, where k is the number of nodes in the residual block (3 in Figure 7). One chain represents the case of computing layer 2 on the mobile and the other one represents the case of computing layer 2 on the cloud. In Figure 7, we have only shown the weights that need to be modified, where D_2 and U_2 are the cost of downloading and uploading the output of layer 2, respectively.

By solving the shortest path problem in the JointDNN graph model, we can obtain the optimal scheduling of inference in DNNs. The online training consists of one inference and one back-propagation step. The total number of layers is noted by N consistently throughout this paper so there are $2N$ layers for modeling training, where the second N layers are the mirrored version of the first N layers, and their associated operations are the gradients of the error function concerning the DNN's weights. The main difference between the mobile cloud computing graph of inference and online training is the need for updating the model by downloading the new weights from the cloud. We assume that the cloud server performs the whole back-propagation step separately, even if it is scheduled to be done on the mobile, therefore, there is no need for the mobile device to upload the weights that are updated by itself to save mobile energy consumption. The modification in the JointDNN graph model is adding the costs of downloading weights of the layers that are updated in the cloud to $ED_{i,j}$. The shortest path problem can be solved in polynomial time

efficiently.

However, the problem of the shortest path subjected to constraints is NP-Complete [22]. For instance, assuming our standard graph is constructed for energy and we need to find the shortest path subject to the constraint of the total latency of that path is less than a time deadline (QoS). However, there is an approximation solution to this problem, "LARAC" algorithm [23], the nature of our application does not require to solve this optimization problem frequently, therefore, we aim to obtain the optimal solution. We can constitute a small look-up table of optimization results for a different set of parameters (e.g. network bandwidth, cloud server load, etc.). We provide the ILP formulations of DNN partitioning in the following sections.

2.3 ILP Setup

2.3.1 Performance Efficient Computation Offloading ILP Setup for Inference

We formulated the scheduling of inference in DNNs as an ILP with tractable number of variables. In our method, first we profile the delay and energy consumption of consecutive layers of size $m \in \{1, 2, \dots, N\}$. Thus, we will have

$$N + (N - 1) + \dots + 1 = N(N + 1)/2 \quad (1)$$

number of different profiling values for delay and energy. Considering layer i to layer j to be computed either on the mobile device or cloud server, we assign two binary

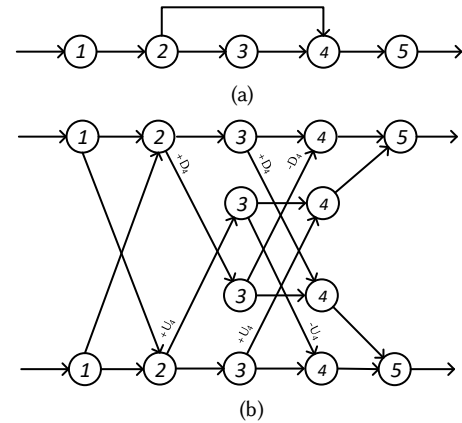


Fig. 7: (a) A residual building block in DNNs(b) Transformation of a residual building block to be able to be used in JointDNN's shortest path based scheduler

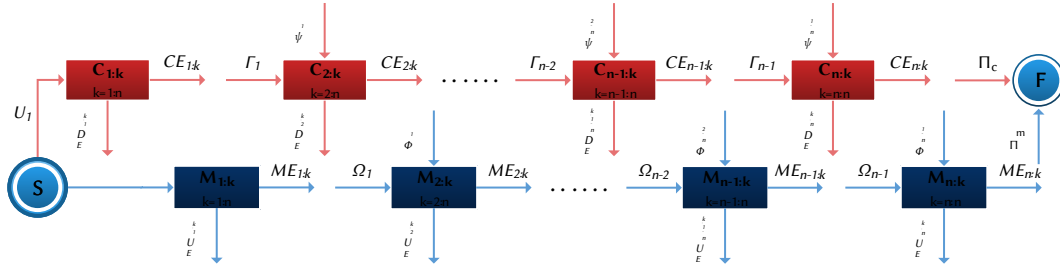


图 6: JointDNN 图模型。从 S 到 F 的最短路径决定了在移动设备或云端执行各层的时间表

仅包括移动计算成本 ($ME_{i,j}$) 2) 从云到云的转换, 仅包括云计算成本 ($CE_{i,j}$) 3) 从移动到云的转换, 包括移动计算成本和下一个节点输入的上传成本 ($EU_{i,j} = ME_{i,j} + UID_{j+1}$) 4) 从云端到移动的转换, 包括云计算成本和下一个节点输入的下载成本 ($ED_{i,j} = CE_{i,j} + DOD_{j+1}$)。根据这个公式, 我们可以将计算调度问题转化为寻找从 S 到 F 的最短路径。残差网络是一类功能强大且易于训练的 DNN 架构 [21]。在残差网络中, 如图 7(a) 所示, 一层的输出被馈送到距离至少为 2 的另一层。因此, 我们需要跟踪源层 (图 7 中的节点 2), 以了解该层是在移动设备还是云端计算的。我们的标准图模型有一个记忆, 即前一层。我们提供了一种方法将此类网络的计算图转换为我们的标准模型——JointDNN 图。

在这方面, 我们添加了两个大小为 $k-1$ 的附加链, 其中 k 是残差块中的节点数 (图 7 中的 3)。一条链代表移动端第二层计算的情况, 另一条链代表云端第二层计算的情况。在图 7 中, 我们仅显示了需要修改的权重, 其中 D_2 和 U_2 分别是下载和上传第 2 层输出的成本。

通过解决 JointDNN 图模型中的最短路径问题, 我们可以获得 DNN 中推理的最优调度。在线训练包括一项推理和一项反向传播步骤。在本文中, 总层数始终用 N 表示, 因此有 2 个 N 层用于建模训练, 其中第二个 N 层是第一个 N 层的镜像版本, 它们的相关操作是与 DNN 权重相关的误差函数的梯度。移动云计算推理图与在线训练之间的主要区别在于需要通过从云端下载新权重来更新模型。我们假设云服务器单独执行整个反向传播步骤, 即使它被安排在移动设备上完成, 因此, 移动设备不需要上传自己更新的权重以节省移动能源消耗。JointDNN 图模型中的修改是将下载云端更新的层权重的成本添加到 $ED_{i,j}$ 。最短路径问题可以在多项式时间内解决

有效地。

然而, 受约束的最短路径问题是 NP 完全问题 [22]。例如, 假设我们的标准图是为能源而构建的, 我们需要找到最短路径, 该路径的总延迟小于时间期限 (QoS)。然而, 这个问题有一个近似解, “LARAC” 算法 [23], 我们的应用程序的性质不需要经常解决这个优化问题, 因此, 我们的目标是获得最优解。我们可以为不同的参数集 (例如网络带宽、云服务器负载等) 构建一个小的优化结果查找表。我们在以下部分中提供 DNN 划分的 ILP 公式。

2.3 ILP 设置

2.3.1 Performance Efficient Computation Offloading ILP Setup for Inference

我们将 DNN 中的推理调度制定为具有易于处理的变量数量的 ILP。在我们的方法中, 首先我们分析大小为 $m \in \{1, 2, \dots, N\}$ 的连续层的延迟和能耗。因此, 我们将有

$$N + (N - 1) + \dots + 1 = N(N + 1)/2 \quad (1)$$

延迟和能量的不同分析值的数量。考虑到层 i 到层 j 要在移动设备或云服务器上计算, 我们分配两个二进制

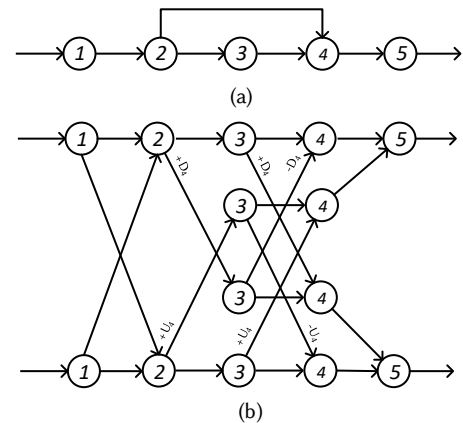


图 7: (a) DNN 中的残差构建块 (b) 将残差构建块转换为能够在 JointDNN 基于最短路径的调度器中使用

variables $m_{i,j}$ and $c_{i,j}$, respectively. Download and upload communication delays needs to be added to the execution time, when switching from/to cloud to/from mobile, respectively.

$$T_{\text{computation}} = \sum_{i=1}^n \sum_{j=i}^n (m_{i,j} \cdot T_{\text{mobile}_{L_{i,j}}} + c_{i,j} \cdot T_{\text{cloud}_{L_{i,j}}}) \quad (2)$$

$$\begin{aligned} T_{\text{communication}} = & \sum_{i=1}^n \sum_{j=i}^n \sum_{k=j+1}^n m_{i,j} \cdot c_{j+1,k} \cdot T_{\text{upload}_{L_j}} \\ & + \sum_{i=1}^n \sum_{j=i}^n \sum_{k=j+1}^n c_{i,j} \cdot m_{j+1,k} \cdot T_{\text{download}_{L_j}} \\ & + \sum_{i=1}^n c_{1,i} \cdot T_{\text{upload}_{L_i}} \\ & + \sum_{i=1}^n c_{i,n} \cdot T_{\text{download}_{L_n}} \end{aligned} \quad (3)$$

$$T_{\text{total}} = T_{\text{computation}} + T_{\text{communication}} \quad (4)$$

$T_{\text{mobile}_{L_{i,j}}}$ and $T_{\text{cloud}_{L_{i,j}}}$ represent the execution time of the i^{th} layer to the j^{th} layer on the mobile and cloud, respectively. $T_{\text{download}_{L_i}}$ and $T_{\text{upload}_{L_i}}$ represent the latency of downloading and uploading the output of the i^{th} layer, respectively. Considering each set of the consecutive layers, whenever $m_{i,j}$ and one of $\{c_{j+1,k}\}_{k=j+1:n}$ are equal to one, the output of the j^{th} layer is uploaded to the cloud. The same argument applies to downloading. We also note that the last two terms in Eq. 3 represent the condition by which the last layer is computed on the cloud and we need to download the output to the mobile device, and the first layer is computed on the cloud and we need to upload the input to the cloud, respectively. To support for residual architectures, we need to add a pair of download and upload terms similar to the first two terms in Eq. 3 for the starting and ending layers of each residual block. In order to guarantee that all layers are computed exactly once, we need to add the following set of constraints:

$$\forall m \in 1 : n : \sum_{i=1}^m \sum_{j=m}^n (m_{i,j} + c_{i,j}) = 1 \quad (5)$$

Because of the non-linearity of multiplication, an additional step is needed to transform Eq. 3 to the standard form of ILP. We define two sets of new variables:

$$\begin{aligned} u_{i,j} &= m_{i,j} \cdot \sum_{k=j+1}^n c_{j+1,k} \\ d_{i,j} &= c_{i,j} \cdot \sum_{k=j+1}^n m_{j+1,k} \end{aligned} \quad (6)$$

with the following constraints:

$$\begin{aligned} u_{i,j} &\leq m_{i,j} \\ u_{i,j} &\leq \sum_{k=j+1}^n c_{j+1,k} \\ m_{i,j} + \sum_{k=j+1}^n c_{j+1,k} - u_{i,j} &\leq 1 \\ d_{i,j} &\leq c_{i,j} \\ d_{i,j} &\leq \sum_{k=j+1}^n m_{j+1,k} \\ c_{i,j} + \sum_{k=j+1}^n m_{j+1,k} - d_{i,j} &\leq 1 \end{aligned} \quad (7)$$

The first two constraints ensure that $u_{i,j}$ will be zero if either $m_{i,j}$ or $\sum_{k=j+1}^n c_{j+1,k}$ are zero. The third inequality guarantees that $u_{i,j}$ will take value one if both binary variables, $m_{i,j}$ and $\sum_{k=j+1}^n c_{j+1,k}$, are set to one. The same reasoning works for $d_{i,j}$. In summary, the total number of variables in our ILP formulation will be $4N(N+1)/2$, where N is total number of layers in the network.

2.3.2 Energy Efficient Computation Offloading ILP Setup for Inference

Because of the nature of the application, we only care about the energy consumption on the mobile side. We formulate ILP as follows:

$$E_{\text{computation}} = \sum_{i=1}^n \sum_{j=i}^n m_{i,j} \cdot E_{\text{mobile}_{L_{i,j}}} \quad (8)$$

$$\begin{aligned} E_{\text{communication}} = & \sum_{i=2}^n \sum_{j=i}^n m_{i,j} \cdot E_{\text{download}_{L_i}} \\ & + \sum_{i=1}^n \sum_{j=i}^{n-1} m_{i,j} \cdot E_{\text{upload}_{L_j}} \\ & + \left(\sum_{i=1}^n (1 - m_{1,i}) - (n-1) \right) \cdot E_{\text{upload}_{L_1}} \\ & + \left(\sum_{i=1}^n (1 - m_{i,n}) - (n-1) \right) \cdot E_{\text{download}_{L_n}} \end{aligned} \quad (9)$$

$$E_{\text{total}} = E_{\text{computation}} + E_{\text{communication}} \quad (10)$$

$E_{\text{mobile}_{L_{i,j}}}$ and $E_{\text{cloud}_{L_{i,j}}}$ represent the amount of energy required to compute the i^{th} layer to the j^{th} layer on the mobile and cloud, respectively. $E_{\text{download}_{L_i}}$ and $E_{\text{upload}_{L_i}}$ represent the energy required to download and upload the output of i^{th} layer, respectively. Similar to performance efficient ILP constraints, each layer should be executed exactly once:

$$\forall m \in 1 : n : \sum_{i=1}^m \sum_{j=m}^n m_{i,j} \leq 1 \quad (11)$$

The ILP problem can be solved for different set of parameters (e.g. different uplink and download speeds),

分别为变量 $m_{i,j}$ 和 $c_{i,j}$ 。当分别从云切换到移动设备时，下载和上传通信延迟需要添加到执行时间中。

$$T_{\text{computation}} = \sum_{i=1}^n \sum_{j=i}^n (m_{i,j} \cdot T_{\text{mobile}_{L_{i,j}}} + c_{i,j} \cdot T_{\text{cloud}_{L_{i,j}}}) \quad (2)$$

$$\begin{aligned} T_{\text{communication}} = & \sum_{i=1}^n \sum_{j=i}^n \sum_{k=j+1}^n m_{i,j} \cdot c_{j+1,k} \cdot T_{\text{upload}_{L_j}} \\ & + \sum_{i=1}^n \sum_{j=i}^n \sum_{k=j+1}^n c_{i,j} \cdot m_{j+1,k} \cdot T_{\text{download}_{L_j}} \\ & + \sum_{i=1}^n c_{1,i} \cdot T_{\text{upload}_{L_i}} \\ & + \sum_{i=1}^n c_{i,n} \cdot T_{\text{download}_{L_n}} \end{aligned} \quad (3)$$

$$T_{\text{total}} = T_{\text{computation}} + T_{\text{communication}} \quad (4)$$

$T_{\text{mobile}_{L_{i,j}}}$, $T_{\text{cloud}_{L_{i,j}}}$ 和 $T_{\text{cloud}_{L_{i,j}}}$ 分别表示移动端和云上 i^{th} 层到 j^{th} 层的执行时间。 $T_{\text{download}_{L_i}}$ 和 $T_{\text{upload}_{L_i}}$ 分别表示下载和上传 i^{th} 层输出的延迟。考虑每组连续层，每当 $m_{i,j}$ 和 $\{c_{j+1,k}\}_{k=j+1:n}$ 之一等于 1 时， j^{th} 层的输出就会上传到云端。同样的论点也适用于下载。我们还注意到方程中的最后两项。3 分别表示最后一层在云端计算，我们需要将输出下载到移动设备，第一层在云端计算，我们需要将输入上传到云端的情况。为了支持残差架构，我们需要添加一对下载和上传项，类似于等式中的前两项。3 为每个残差块的起始层和结束层。为了保证所有层都只计算一次，我们需要添加以下一组约束：

$$\forall m \in 1:n : \sum_{i=1}^m \sum_{j=m}^n (m_{i,j} + c_{i,j}) = 1 \quad (5)$$

由于乘法的非线性，需要一个额外的步骤来转换方程。3 ILP 的标准形式。我们定义两组新变量：

$$\begin{aligned} u_{i,j} &= m_{i,j} \cdot \sum_{k=j+1}^n c_{j+1,k} \\ d_{i,j} &= c_{i,j} \cdot \sum_{k=j+1}^n m_{j+1,k} \end{aligned} \quad (6)$$

具有以下限制：

$$\begin{aligned} u_{i,j} &\leq m_{i,j} \\ u_{i,j} &\leq \sum_{k=j+1}^n c_{j+1,k} \\ m_{i,j} + \sum_{k=j+1}^n c_{j+1,k} - u_{i,j} &\leq 1 \\ d_{i,j} &\leq c_{i,j} \\ d_{i,j} &\leq \sum_{k=j+1}^n m_{j+1,k} \\ c_{i,j} + \sum_{k=j+1}^n m_{j+1,k} - d_{i,j} &\leq 1 \end{aligned} \quad (7)$$

前两个约束确保如果 $m_{i,j}$ 或 $\sum_{l=j+1}^n c_{j+1,l}$ 为零，则 $u_{i,j}$ 为零。第三个不等式保证，如果二元变量 $m_{i,j}$ 和 $\sum_{l=j+1}^n c_{j+1,l}$ 都设置为 1，则 $u_{i,j}$ 的值为 1。同样的推理也适用于 $d_{i,j}$ 。总之，我们的 ILP 公式中的变量总数将为 $4N(N+1)/2$ ，其中 N 是网络中的总层数。

2.3.2 Energy Efficient Computation Offloading ILP Setup for Inference

由于应用的性质，我们只关心移动端的能耗。我们将 ILP 制定如下：

$$E_{\text{computation}} = \sum_{i=1}^n \sum_{j=i}^n m_{i,j} \cdot E_{\text{mobile}_{L_{i,j}}} \quad (8)$$

$$\begin{aligned} E_{\text{communication}} = & \sum_{i=2}^n \sum_{j=i}^n m_{i,j} \cdot E_{\text{download}_{L_i}} \\ & + \sum_{i=1}^n \sum_{j=i}^{n-1} m_{i,j} \cdot E_{\text{upload}_{L_j}} \\ & + \left(\sum_{i=1}^n (1 - m_{1,i}) - (n-1) \right) \cdot E_{\text{upload}_{L_1}} \\ & + \left(\sum_{i=1}^n (1 - m_{i,n}) - (n-1) \right) \cdot E_{\text{download}_{L_n}} \end{aligned} \quad (9)$$

$$E_{\text{total}} = E_{\text{computation}} + E_{\text{communication}} \quad (10)$$

$E_{\text{mobile}_{L_{i,j}}}$ 和 $E_{\text{cloud}_{L_{i,j}}}$ 分别表示在移动设备和云端计算 i^{th} 层到 j^{th} 层所需的能量。 $E_{\text{download}_{L_i}}$ 和 $E_{\text{upload}_{L_i}}$ 分别表示下载和上传 i^{th} 层输出所需的能量。与性能高效的 ILP 约束类似，每一层应该只执行一次：

$$\forall m \in 1:n : \sum_{i=1}^m \sum_{j=m}^n m_{i,j} \leq 1 \quad (11)$$

ILP 问题可以针对不同的参数集（例如不同的上行链路和下载速度）来解决，

and then the scheduling results can be stored as a look-up table in the mobile device. Moreover because the number of variables in this setup is tractable solving ILP is quick. For instance, solving ILP for AlexNet takes around 0.045 seconds on Intel(R) Core(TM) i7-3770 CPU with MATLAB®'s `intlinprog()` function using primal simplex algorithm.

2.3.3 Performance Efficient Computation Offloading ILP Setup for Training

The ILP formulation of online training phase is very similar to that of inference. In online training we have $2N$ layers instead of N obtained by mirroring the DNN, where the second N layers are backward propagation. Moreover, we need to download the weights that are updated in the cloud to the mobile. We assume that the cloud server always has the most updated version of the weights and does not require the mobile device to upload the updated weights. The following terms need to be added for the ILP setup of training:

$$T_{\text{computation}} = \sum_{i=1}^{2n} \sum_{j=i}^{2n} (m_{i,j} \cdot T_{\text{mobile}_{L_{i,j}}} + c_{i,j} \cdot T_{\text{cloud}_{L_{i,j}}}) \quad (12)$$

$$\begin{aligned} T_{\text{communication}} = & \sum_{i=1}^{2n} \sum_{j=i}^{2n} \sum_{k=j+1}^{2n} m_{i,j} \cdot c_{j+1,k} \cdot T_{\text{upload}_{L_j}} \\ & + \sum_{i=1}^{2n} \sum_{j=i}^{2n} \sum_{k=j+1}^{2n} c_{i,j} \cdot m_{j+1,k} \cdot T_{\text{download}_{L_j}} \\ & + \sum_{i=1}^n c_{1,i} \cdot T_{\text{upload}_{L_i}} \\ & + \sum_{i=n+1}^{2n} \sum_{j=i}^{2n} c_{i,j} \cdot T_{\text{download}_{W_i}} \end{aligned} \quad (13)$$

$$T_{\text{total}} = T_{\text{computation}} + T_{\text{communication}} \quad (14)$$

2.3.4 Energy Efficient Computation Offloading ILP Setup for Training

$$E_{\text{computation}} = \sum_{i=1}^{2n} \sum_{j=i}^{2n} m_{i,j} \cdot E_{\text{mobile}_{L_{i,j}}} \quad (15)$$

$$\begin{aligned} E_{\text{communication}} = & \sum_{i=2}^{2n} \sum_{j=i}^{2n} m_{i,j} \cdot E_{\text{download}_{L_i}} \\ & + \sum_{i=1}^{2n} \sum_{j=i}^{2n-1} m_{i,j} \cdot E_{\text{upload}_{L_j}} \\ & + \left(\sum_{i=1}^{2n} (1 - m_{1,i}) - (2n - 1) \right) \cdot E_{\text{upload}_{L_1}} \\ & + \left(\sum_{i=n+1}^{2n} \sum_{j=i}^{2n} (1 - m_{i,j}) - (n - 1) \right) \cdot E_{\text{download}_{W_i}} \end{aligned} \quad (16)$$

$$E_{\text{total}} = E_{\text{computation}} + E_{\text{communication}} \quad (17)$$

2.3.5 Scenarios

There can be different optimization scenarios defined for ILP as listed below:

- **Performance efficient computation:** In this case, it is sufficient to solve the ILP formulation for performance efficient computation offloading.
- **Energy efficient computation:** In this case, it is sufficient to solve the ILP formulation for energy efficient computation offloading.
- **Battery budget limitation:** In this case, based on the available battery, the operating system can decide to dedicate a specific amount of energy consumption to each application. By adding the following constraint to the performance efficient ILP formulation, our framework would adapt to battery limitations:

$$E_{\text{computation}} + E_{\text{communication}} \leq E_{\text{ubound}} \quad (18)$$

- **Cloud limited resources:** In the presence of cloud server congestion or limitations on user's subscription, we can apply execution time constraints to each application to alleviate the server load:

$$\sum_{i=1}^n \sum_{j=i}^n c_{i,j} \cdot T_{\text{cloud}_{L_{i,j}}} \leq T_{\text{ubound}} \quad (19)$$

- **QoS:** In this scenario, we minimize the required energy consumption while meeting a specified deadline:

$$\begin{aligned} & \min \{ E_{\text{computation}} + E_{\text{communication}} \} \\ & T_{\text{computation}} + T_{\text{communication}} \leq T_{\text{QoS}} \end{aligned} \quad (20)$$

This constraint could be applied to both energy and performance efficient ILP formulations.

3 EVALUATION

3.1 Deep Architecture Benchmarks

Since the architecture of neural networks depends on the type of application, we have chosen three common application types of DNNs as shown in Table 2:

- 1) **Discriminative neural networks** are a class of models in machine learning for modeling the conditional probability distribution $P(y|x)$. This class generally is used in classification and regression tasks. AlexNet [24], OverFeat [25], VGG16 [26], Deep Speech [27], ResNet [21], and NiN [28] are well-known discriminative models we use as benchmarks in this experiment. Except for Deep Speech, used for speech recognition, all other benchmarks are used in image classification tasks.
- 2) **Generative neural networks** model the joint probability distribution $P(x, y)$, allowing generation of new samples. These networks have applications in Computer Vision [29] and Robotics [30], which can be deployed on a mobile device. Chair [31] is a

然后调度结果可以作为查找表存储在移动设备中。此外，由于此设置中的变量数量很容易处理，因此求解 ILP 很快。例如，在 Intel(R) Core(TM) i7-3770 CPU 上，使用原始单纯形算法，使用 MATLAB® 的 intlinprog() 函数求解 AlexNet 的 ILP 大约需要 0.045 秒。

2.3.3 Performance Efficient Computation Offloading ILP Setup for Training

在线训练阶段的 ILP 表述与推理非常相似。在在线训练中，我们有 $2N$ 层，而不是通过镜像 DNN 获得的 N 层，其中第二个 N 层是反向传播。此外，我们还需要将云端更新的权重下载到手机上。我们假设云服务器始终拥有最新版本的权重，并且不需要移动设备上传更新的权重。需要为训练的 ILP 设置添加以下术语：

$$T_{\text{computation}} = \sum_{i=1}^{2n} \sum_{j=i}^{2n} (m_{i,j} \cdot T_{\text{mobile}_{L_{i,j}}} + c_{i,j} \cdot T_{\text{cloud}_{L_{i,j}}}) \quad (12)$$

$$\begin{aligned} T_{\text{communication}} = & \sum_{i=1}^{2n} \sum_{j=i}^{2n} \sum_{k=j+1}^{2n} m_{i,j} \cdot c_{j+1,k} \cdot T_{\text{upload}_{L_j}} \\ & + \sum_{i=1}^{2n} \sum_{j=i}^{2n} \sum_{k=j+1}^{2n} c_{i,j} \cdot m_{j+1,k} \cdot T_{\text{download}_{L_j}} \\ & + \sum_{i=1}^n c_{1,i} \cdot T_{\text{upload}_{L_i}} \\ & + \sum_{i=n+1}^{2n} \sum_{j=i}^{2n} c_{i,j} \cdot T_{\text{download}_{W_i}} \end{aligned} \quad (13)$$

$$T_{\text{total}} = T_{\text{computation}} + T_{\text{communication}} \quad (14)$$

2.3.4 Energy Efficient Computation Offloading ILP Setup for Training

$$E_{\text{computation}} = \sum_{i=1}^{2n} \sum_{j=i}^{2n} m_{i,j} \cdot E_{\text{mobile}_{L_{i,j}}} \quad (15)$$

$$\begin{aligned} E_{\text{communication}} = & \sum_{i=2}^{2n} \sum_{j=i}^{2n} m_{i,j} \cdot E_{\text{download}_{L_i}} \\ & + \sum_{i=1}^{2n} \sum_{j=i}^{2n-1} m_{i,j} \cdot E_{\text{upload}_{L_j}} \\ & + \left(\sum_{i=1}^{2n} (1 - m_{1,i}) - (2n - 1) \right) \cdot E_{\text{upload}_{L_1}} \\ & + \left(\sum_{i=n+1}^{2n} \sum_{j=i}^{2n} (1 - m_{i,j}) - (n - 1) \right) \cdot E_{\text{download}_{W_i}} \end{aligned} \quad (16)$$

$$E_{\text{total}} = E_{\text{computation}} + E_{\text{communication}} \quad (17)$$

2.3.5 Scenarios

可以为 ILP 定义不同的优化方案，如下所列：

- 性能高效计算：在这种情况下，解决 ILP 公式就足以实现性能高效的计算。
- 能源效率计算：在这种情况下，求解 ILP 公式就足以实现负载的能源效率计算。
- 电池预算限制：在这种情况下，根据可用电池，操作系统可以决定为每个应用程序分配特定的能耗量。通过在性能高效的 ILP 公式中添加以下约束，我们的框架将适应电池限制：

$$E_{\text{computation}} + E_{\text{communication}} \leq E_{\text{ubound}} \quad (18)$$

- 云资源有限：在云服务器拥塞或用户订阅受到限制的情况下，我们可以对每个应用程序应用执行时间限制以减轻服务器负载：

$$\sum_{i=1}^n \sum_{j=i}^n c_{i,j} \cdot T_{\text{cloud}_{L_{i,j}}} \leq T_{\text{ubound}} \quad (19)$$

- QoS：在这种情况下，我们在满足指定期限的同时最大限度地减少所需的能源消耗：

$$\min \{ E_{\text{computation}} + E_{\text{communication}} \} \quad (20)$$

$T_{\text{computation}} + T_{\text{communication}} \leq T_{\text{QoS}}$
该约束可应用于能源和性能高效的 ILP 配方。

3 评价

3.1 深层架构基准测试

由于神经网络的架构取决于应用类型，因此我们选择了三种常见的 DNN 应用类型，如表 2 所示：

1) 判别神经网络是机器学习中的一类模型，用于对条件概率分布 $P(y|x)$ 进行建模。该类通常用于分类和回归任务。AlexNet [24]、OverFeat [25]、VGG16 [26]、Deep Speech [27]、ResNet [21] 和 NiN [28] 是我们在本实验中用作基准的著名判别模型。除了用于语音识别的 Deep Speech 之外，所有其他基准均用于图像分类任务。

2) 生成神经网络对联合概率分布 $P(x, y)$ 进行建模，允许生成新样本。这些网络在计算机视觉 [29] 和机器人技术 [30] 中有应用，可以部署在移动设备上。椅子 [31] 是

Algorithm 1: JointDNN engine optimal scheduling of DNNs

```

1 function JointDNN ( $N, L_i, D_i, NB, NP$ );
  Input : 1:  $N$ : number of layers in the DNN
          2:  $L_i | i = 1 : N$ : layers in the DNN
          3:  $D_i | i = 1 : N$ : data size at each layer
          4:  $NB$ : mobile network bandwidth
          5:  $NP$ : mobile network uplink and downlink
             power consumption
  Output: Optimal schedule of DNN
2 for  $i = 0; i < N; i = i + 1$  do
3   for  $j = 0; j < N; j = j + 1$  do
4      $Latency_{i,j}, Energy_{i,j} =$ 
       ProfileGroupedLayers( $i, j$ );
5   end
6 end
7  $G, S, F =$ 
   ConstructShortestPathGraph( $N, L_i, D_i, NB, NP$ ) //  $S$ 
   and  $F$  are start and finish nodes and  $G$  is the
   JointDNN graph model
8 if no constraints then
9    $schedule = \text{ShortestPath}(G, S, F)$ 
10 else
11   if Battery Limited Constraint then
12      $E_{comm} + E_{comp} \leq E_{ubound}$ 
13      $schedule =$ 
       PerformanceEfficientILP( $N, L_i, D_i, NB, NP$ )
14   end
15   if Cloud Server Constraint then
16      $\sum_{i=1}^n \sum_{j=i}^n c_{i,j} \cdot T_{cloud, L_{i,j}} \leq T_{ubound}$ 
17      $schedule =$ 
       PerformanceEfficientILP( $N, L_i, D_i, NB, NP$ )
18   end
19   if QoS then
20      $T_{comm} + T_{comp} \leq T_{QoS}$ 
21      $schedule =$ 
       EnergyEfficientILP( $N, L_i, D_i, NB, NP$ )
22   end
23   ;
24 end
25 return  $schedule$ ;

```

TABLE 2: Benchmark Specifications

Type	Model	Layers
Discriminative	AlexNet	21
	OverFeat	14
	Deep Speech	10
	ResNet	70
	VGG16	37
	NiN	29
Generative	Chair	10
Autoencoder	Pix2Pix	32

generative model we use as a benchmark in this work.

- 3) **Autoencoders** are another class of neural networks used to learn a representation for a data set. Their applications are image reconstruction, image to image translation, and denoising to name a few. Mobile robots can be equipped with autoencoders to be used in their computer vision tasks. We use Pix2Pix [32], as a benchmark from this class.

TABLE 3: Mobile networks specifications in the U.S.

Param.	3G	4G	Wi-Fi
Download speed (Mbps)	2.0275	13.76	54.97
Upload speed (Mbps)	1.1	5.85	18.88
α_u (mW/Mbps)	868.98	438.39	283.17
α_d (mW/Mbps)	122.12	51.97	137.01
β (mW)	817.88	1288.04	132.86

3.2 Mobile and Server Setup

We used the Jetson TX2 module developed by NVIDIA® [33], a fair representation of mobile computation power as our mobile device. This module enables efficient implementation of DNN applications used in products such as robots, drones, and smart cameras. It is equipped with NVIDIA Pascal® GPU with 256 CUDA cores and a shared 8 GB 128 bit LPDDR4 memory between GPU and CPU. To measure the power consumption of the mobile platform, we used INA226 power sensor [34].

NVIDIA® Tesla® K40C [35] with 12 GB memory serves as our server GPU. The computation capability of this device is more than one order of magnitude compared to our mobile device.

3.3 Communication Parameters

To model the communication between platforms, we used the average download and upload speed of mobile Internet [36], [37] for different networks (3G, 4G and Wi-Fi) as shown in Table 3.

The communication power for download (P_d) and upload (P_u) is dependent on the network throughput (t_d and t_u). Comprehensive examinations in [38] indicates that uplink and downlink power can be modeled with linear equations (Eq. 21) fairly accurate with less than 6% error rate. Table 3 shows the parameter values of this equation for different networks.

$$\begin{aligned}
 P_u &= \alpha_u t_u + \beta \\
 P_d &= \alpha_d t_d + \beta
 \end{aligned} \tag{21}$$

4 RESULTS

The latency and energy improvements of inference and online training with our engine for 8 different benchmarks are shown in Figures 8 and 9, respectively. We considered the best case of mobile-only and cloud-only as our baseline. JointDNN can achieve up to 66% and 86% improvements in latency and energy consumption, respectively during inference. Communication cost increases linearly with batch size while this is not the case for computation cost and it grows with a much lower rate, as depicted in 10(b). Therefore, a key observation is that as we increase the batch size, the mobile-only approach becomes more preferable.

During online training, the huge communication overhead of transmitting the updated weights will be added to the total cost. Therefore, to avoid downloading this large data, only a few back-propagation steps are computed in the cloud server. We performed a simulation by varying the percentage of updated weight. As the percentage of updated weights increases, the latency and energy consumption becomes constant which is shown in Figure 10. This is the result of the fact that all the backpropagations will be performed on the mobile device and weights are

Algorithm 1: JointDNN engine optimal scheduling of DNNs

```

1 function JointDNN ( $N, L_i, D_i, NB, NP$ );
  Input : 1:  $N$ : number of layers in the DNN
          2:  $L_i | i = 1 : N$ : layers in the DNN
          3:  $D_i | i = 1 : N$ : data size at each layer
          4:  $NB$ : mobile network bandwidth
          5:  $NP$ : mobile network uplink and downlink
           power consumption
  Output: Optimal schedule of DNN
2 for  $i = 0; i < N; i = i + 1$  do
3   for  $j = 0; j < N; j = j + 1$  do
4      $Latency_{i,j}, Energy_{i,j} =$ 
       ProfileGroupedLayers( $i, j$ );
5   end
6 end
7  $G, S, F =$ 
   ConstructShortestPathGraph( $N, L_i, D_i, NB, NP$ ) //  $S$ 
   and  $F$  are start and finish nodes and  $G$  is the
   JointDNN graph model
8 if no constraints then
9    $schedule = \text{ShortestPath}(G, S, F)$ 
10 else
11   if Battery Limited Constraint then
12      $E_{comm} + E_{comp} \leq E_{ubound}$ 
13      $schedule =$ 
       PerformanceEfficientILP( $N, L_i, D_i, NB, NP$ )
14   end
15   if Cloud Server Constraint then
16      $\sum_{i=1}^n \sum_{j=i}^n c_{i,j} \cdot T_{cloud, L_{i,j}} \leq T_{ubound}$ 
17      $schedule =$ 
       PerformanceEfficientILP( $N, L_i, D_i, NB, NP$ )
18   end
19   if QoS then
20      $T_{comm} + T_{comp} \leq T_{QoS}$ 
21      $schedule =$ 
       EnergyEfficientILP( $N, L_i, D_i, NB, NP$ )
22   end
23   ;
24 end
25 return  $schedule$ ;

```

表 2: 基准规格

Type	Model	Layers
Discriminative	AlexNet	21
	OverFeat	14
	Deep Speech	10
	ResNet	70
	VGG16	37
	NiN	29
Generative	Chair	10
Autoencoder	Pix2Pix	32

我们在这项工作中使用生成模型作为基准。

3) 自动编码器是另一类用于学习数据集表示的神经网络。它们的应用包括图像重建、图像到图像转换和去噪等。移动机器人可以配备自动编码器，用于计算机视觉任务。我们使用 Pix2Pix [32] 作为此类的基准。

表 3: 美国的移动网络规格

Param.	3G	4G	Wi-Fi
Download speed (Mbps)	2.0275	13.76	54.97
Upload speed (Mbps)	1.1	5.85	18.88
α_u (mW/Mbps)	868.98	438.39	283.17
α_d (mW/Mbps)	122.12	51.97	137.01
β (mW)	817.88	1288.04	132.86

3.2 移动端和服务端设置

我们使用 NVIDIA® [33] 开发的 Jetson TX2 模块，它公平地代表了我们的移动设备的移动计算能力。该模块可以有效实现机器人、无人机和智能相机等产品中使用的 DNN 应用程序。它配备了 NVIDIA Pascal® GPU，具有 256 个 CUDA 核心，GPU 和 CPU 之间共享 8 GB 128 位 LPDDR4 内存。为了测量移动平台的功耗，我们使用了 INA226 功率传感器[34]。

具有 12 GB 内存的 NVIDIA® Tesla® K40C [35] 作为我们的服务器 GPU。与我们的移动设备相比，该设备的计算能力高出一个数量级以上。

3.3 通讯参数

为了对平台之间的通信进行建模，我们使用了不同网络（3G、4G 和 Wi-Fi）的移动互联网 [36]、[37] 的平均下载和上传速度，如表 3 所示。

下载 (P_d) 和上传 (P_u) 的通信功率取决于网络吞吐量 (t_d 和 t_u)。[38] 中的综合检查表明，上行链路和下行链路功率可以使用线性方程（方程 21）进行建模，相当准确，误差率小于 6%。表 3 显示了该方程针对不同网络的参数值。

$$\begin{aligned}
 P_u &= \alpha_u t_u + \beta \\
 P_d &= \alpha_d t_d + \beta
 \end{aligned} \tag{21}$$

4 个结果

图 8 和图 9 分别显示了我们的引擎针对 8 个不同基准的推理和在线训练的延迟和能量改进。我们将纯移动和纯云的最佳情况视为我们的基准。JointDNN 在推理过程中可以分别将延迟和能耗提高 66% 和 86%。通信成本随着批量大小线性增加，而计算成本则不然，它的增长速度要低得多，如图 10(b) 所示。因此，一个关键的观察结果是，随着我们增加批量大小，仅移动设备的方法变得更可取。

在在线训练期间，传输更新权重的巨大通信开销将添加到总成本中。因此，为了避免下载如此大的数据，在云服务器中仅计算一些反向传播步骤。我们通过改变更新权重的百分比来进行模拟。随着更新权重百分比的增加，延迟和能耗变得恒定，如图 10 所示。这是因为所有反向传播都将在移动设备上执行，并且权重为

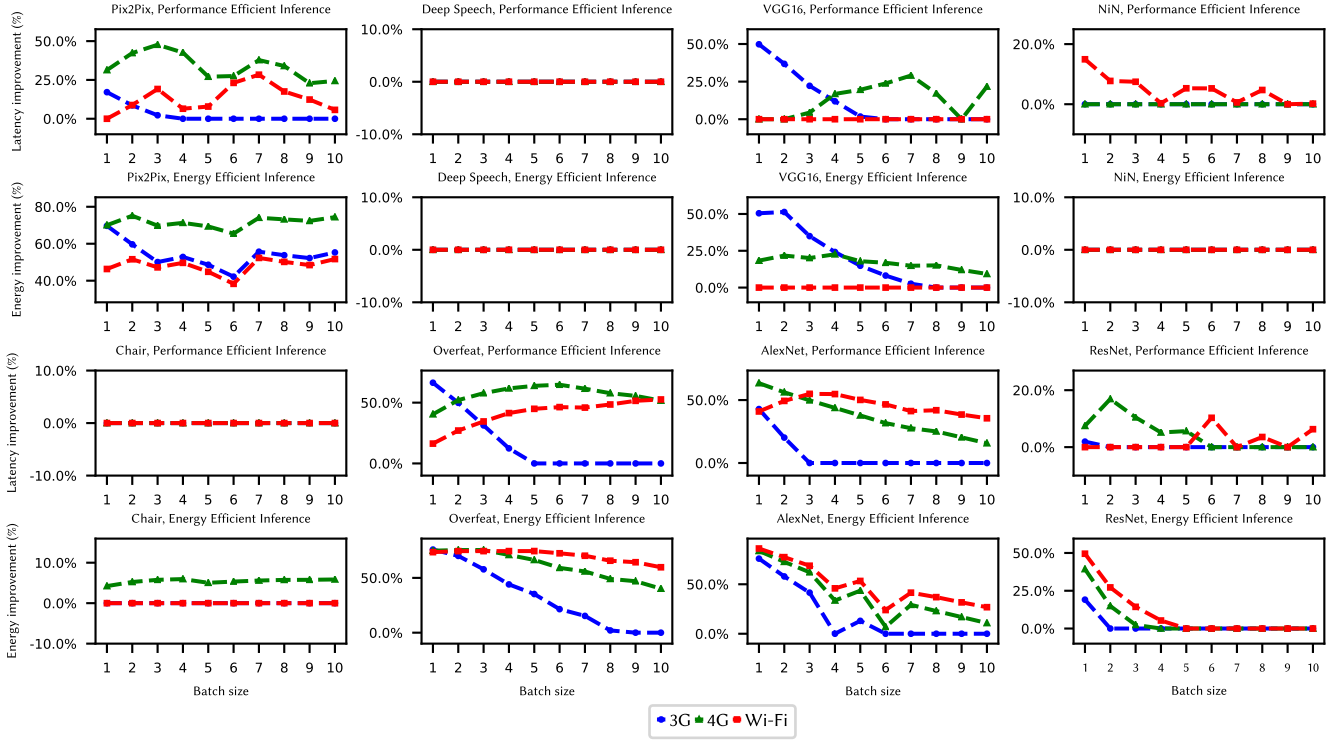


Fig. 8: Latency and energy improvements for different batch sizes during inference over the base case of mobile-only and cloud-only approaches.

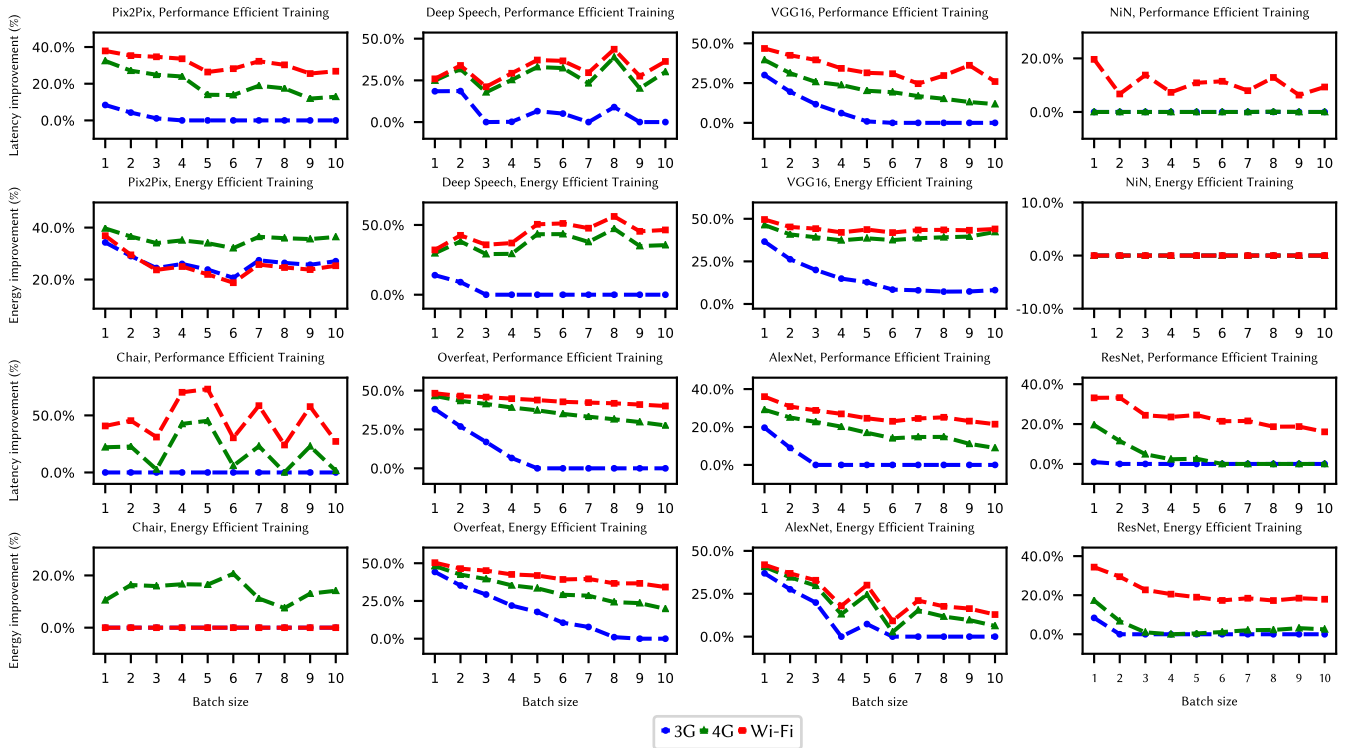
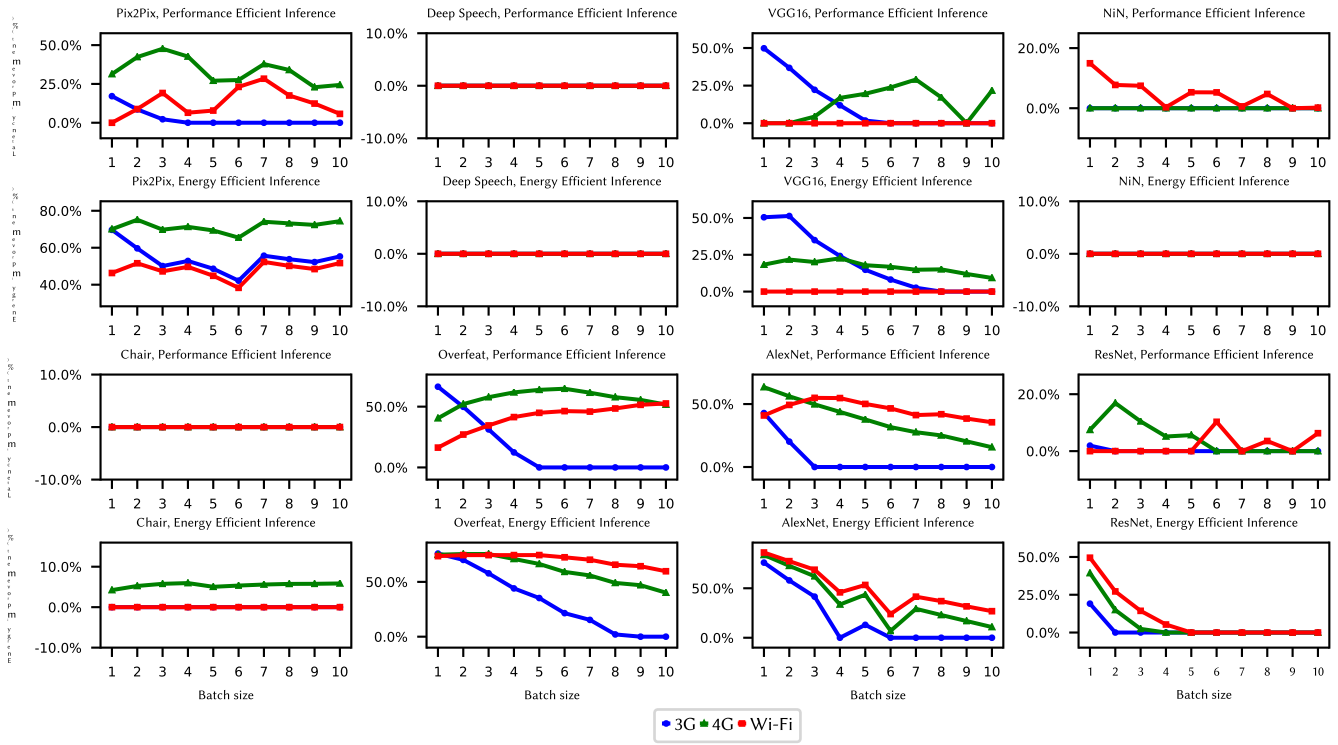


Fig. 9: Latency and energy improvements for different batch sizes during training over the base case of mobile-only and cloud-only approaches.



如图图 8：在仅移动和仅云方法的基本情况下进行推理时，不同批量大小的延迟和能耗得到改善。

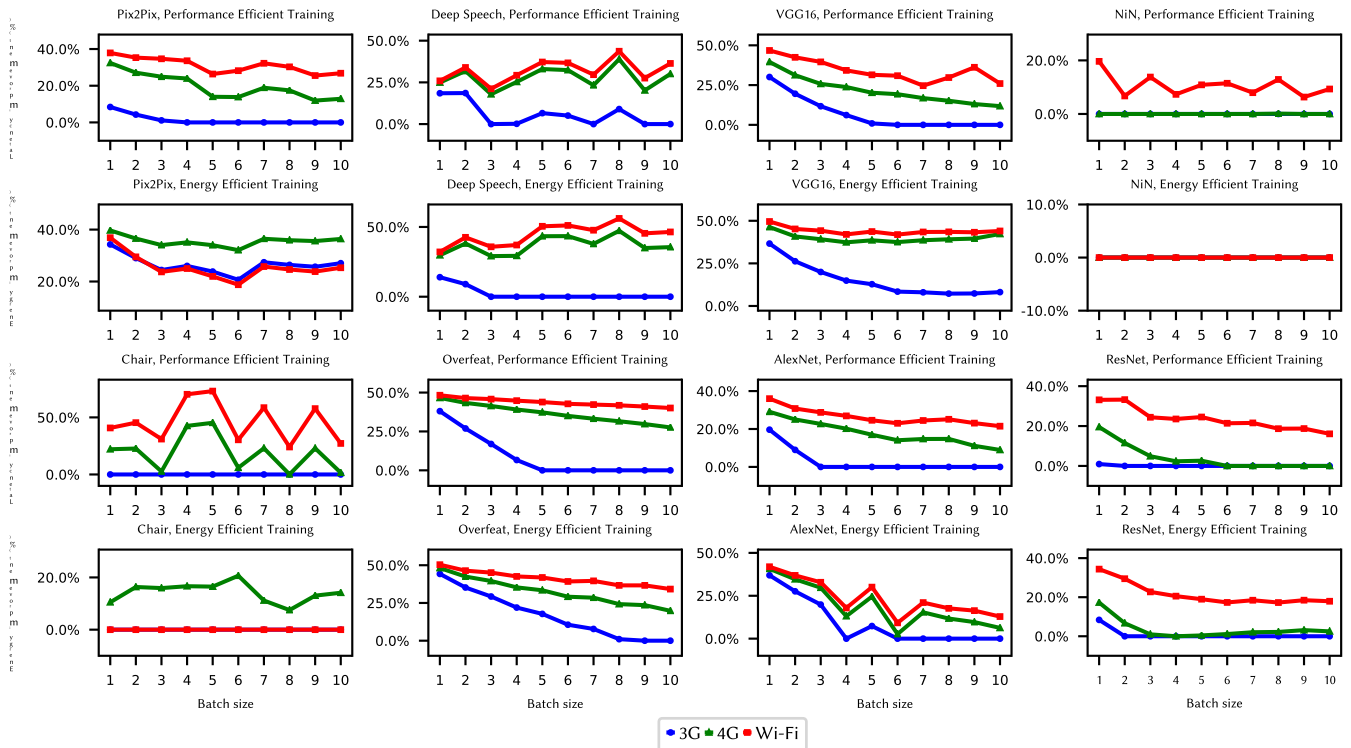


图 9：在仅移动设备的基本情况下训练期间，不同批量大小的延迟和能量改进仅云方法。

not transferred from the cloud to the mobile. JointDNN can achieve improvements up to 73% in latency and 56% in energy consumption during inference.

Different patterns of scheduling are demonstrated in Figure 11. They represent the optimal solution in the Wi-Fi network while optimizing for latency while mobile/cloud is allowed to use up to half of their computing resources. They show how the computations in DNN is divided between the mobile and the cloud. As can be seen, discriminative models (e.g. AlexNet), inference follows a mobile-cloud pattern and training follows a mobile-cloud-mobile pattern. The intuition is that the last layers are computationally intensive (fully connected layers) but with small data sizes, which require a low communication cost, therefore, the last layers tend to be computed on the cloud. For generative models (e.g. Chair), the execution schedule of inference is the opposite of discriminative networks, in which the last layers are generally huge and in the optimal solution they are computed on the mobile. The reason behind not having any improvement over the base case of mobile-only is that the amount of transferred data is large. Besides, cloud-only becomes the best solution when the amount of transferred data is small (e.g. generative models). Lastly, for autoencoders, where both the input and output data sizes are large, the first and last layers are computed on the mobile.

JointDNN pushes some parts of the computations toward the mobile device. As a result, this will lead to less workload on the cloud server. As we see in Table 4, we can reduce the cloud server’s workload up to 84% and 53% on average, which enables the cloud provider to provide service to more users, while obtaining higher performance and lower energy consumption compared to single-platform approaches.

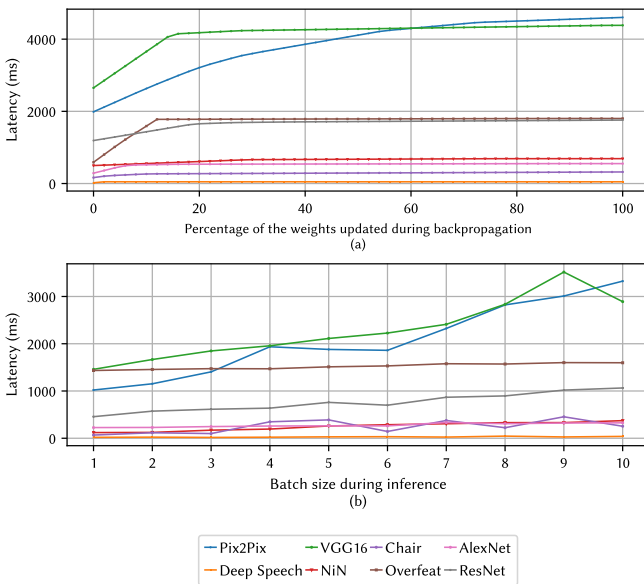


Fig. 10: (a) Latency of one epoch of online training using JointDNN algorithm vs percentage of updated weights (b) Latency of mobile-only inference vs. batch size.

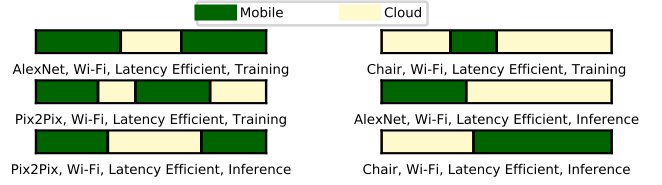


Fig. 11: Interesting schedules of execution for three types of DNN architectures while mobile/cloud are allowed to use up to half of their computing resources.

TABLE 4: Workload reduction of the cloud server in different mobile networks

Optimization Target	3G (%)	4G (%)	Wi-Fi (%)
Latency	84	49	12
Energy	73	49	51

4.1 Communication Dominance

Execution time and energy breakdown for AlexNet, which is noted as a representative for the state-of-the-art architectures deployed in cloud servers, is depicted in Figure 12. The cloud-only approach is dominated by the communication costs. As demonstrated in Figure 12, 99%, 93% and 81% of the total execution time are used for communication in case of 3G, 4G, and Wi-Fi, respectively. This relative portion also applies to energy consumption. Comparing the latency and energy of the communication to those of mobile-only approach, we notice that the mobile-only approach for AlexNet is better than the cloud-only approach in all the mobile networks. We apply loss-less compression methods to reduce the overheads of communication, which will be covered in the next section.

4.2 Layer Compression

The preliminary results of our experiments show that more than 75% of the total energy and delay cost in DNNs are caused by communication in the collaborative approach. This cost is directly proportional to the size of the layer being downloaded to or uploaded from the mobile device. Because of the complex feature extraction process of DNNs, the size of some of the intermediate layers are even larger than the network’s input data. For example, this ratio can go as high as $10\times$ in VGG16. To address this bottleneck, we investigated the compression of the feature data before any communication. This process can be applied to different DNN architecture types; however, we only considered CNNs due to their specific characteristics explained later in detail.

CNN architectures are mostly used for image and video recognition applications. Because of the spatially local preservation characteristics of *conv* layers, we can assume that the outputs of the first convolution layers are following the same structure as the input image, as shown in Figure 13. Moreover, a big ratio of layer outputs is expected to be zero due to the presence of the ReLU layer. Our observations shows that the ratio of neurons equal to zero (ZR) varies from 50% to 90% after *relu* in CNNs. These two characteristics, layers being similar to the input image, and a large proportion of their data being a single value, suggest that we can employ existing image compression techniques to their output.

没有从云端传输到移动设备。JointDNN 可以在推理过程中实现高达 73% 的延迟改进和 56% 的能耗改进。

图 11 展示了不同的调度模式。它们代表 Wi-Fi 网络中的最佳解决方案，同时优化延迟，同时允许移动/云使用最多一半的计算资源。它们展示了 DNN 中的计算如何在移动设备和云端之间划分。可以看出，判别模型（例如 AlexNet）的推理遵循移动-云模式，训练遵循移动-云-移动模式。直觉是，最后一层是计算密集型的（全连接层），但数据量较小，需要较低的通信成本，因此，最后一层往往在云端计算。对于生成模型（例如 Chair），推理的执行时间表与判别网络相反，其中最后一层通常很大，并且在最佳解决方案中它们是在移动设备上计算的。与仅限移动设备的基本情况相比没有任何改进的原因是传输的数据量很大。此外，当传输的数据量较小（例如生成模型）时，仅云成为最佳解决方案。最后，对于自动编码器，输入和输出数据量都很大，第一层和最后一层是在移动设备上计算的。

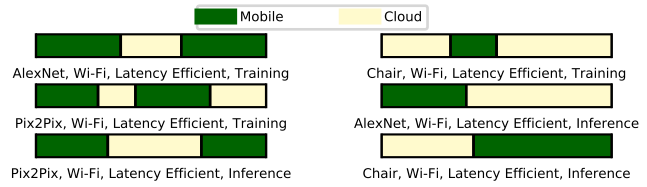


图 11：三种类型的 DNN 架构的有趣的执行计划，同时允许移动/云使用最多一半的计算资源。

表 4：不同移动网络中云服务器的工作负载减少

Optimization Target	3G (%)	4G (%)	Wi-Fi (%)
Latency	84	49	12
Energy	73	49	51

4.1 通讯优势

AlexNet 被认为是部署在云服务器中的最先进架构的代表，其执行时间和能源细分如图 12 所示。纯云方法主要由通信成本决定。如图 12 所示，在 3G、4G 和 Wi-Fi 情况下，总执行时间的 99%、93% 和 81% 分别用于通信。这一相对比例也适用于能源消耗。将通信延迟和能量与仅移动方法的通信延迟和能量进行比较，我们注意到 AlexNet 的仅移动方法优于所有移动网络中的仅云方法。我们应用无损压缩方法来减少通信开销，这将在下一节中介绍。

JointDNN 将部分计算推向移动设备。因此，这将导致云服务器的工作负载减少。正如表 4 所示，我们可以将云服务器的工作负载平均减少 84% 和 53%，这使得云提供商能够为更多用户提供服务，同时与单平台方法相比获得更高的性能和更低的能耗。

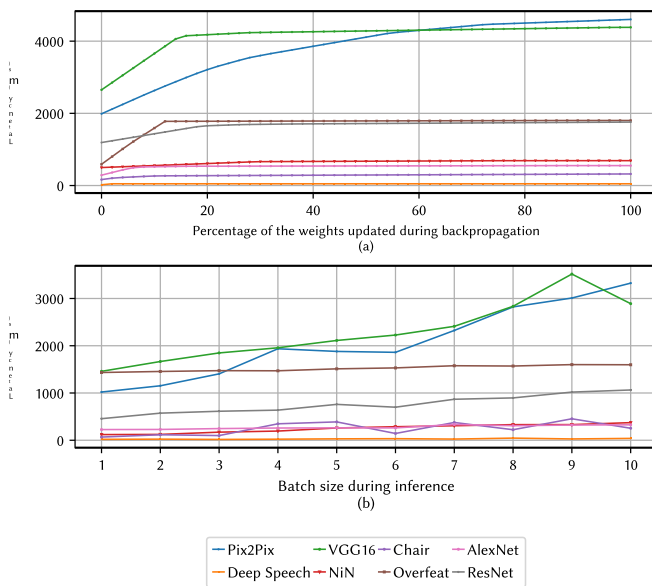


图 10：(a) 使用 JointDNN 算法进行在线训练的一个周期的延迟与更新权重的百分比的关系 (b) 仅移动设备推理的延迟与批量大小的关系。

4.2 层压缩

我们实验的初步结果表明，DNN 中超过 75% 的总能量和延迟成本是由协作方法中的通信引起的。此成本与从移动设备下载或上传的层的大小成正比。由于 DNN 特征提取过程复杂，一些中间层的大小甚至大于网络的输入数据。例如，在 VGG16 中，该比率可以高达 10 $\times$ 。为了解决这个瓶颈，我们在任何通信之前研究了特征数据的压缩。这个过程可以应用于不同的 DNN 架构类型；然而，我们只考虑了 CNN，因为它们的具体特征稍后会详细解释。

CNN 架构主要用于图像和视频识别应用。由于 conv 层的空间局部保存特性，我们可以假设第一个卷积层的输出遵循与输入图像相同的结构，如图 13 所示。此外，由于 ReLU 层的存在，预计层输出的很大比例为零。我们的观察表明，CNN 中 relu 之后为零 (ZR) 的神经元比例从 50% 到 90% 不等。这两个特征，即层与输入图像相似，以及它们的大部分数据是单个值，表明我们可以将现有的图像压缩技术应用于它们的输出。

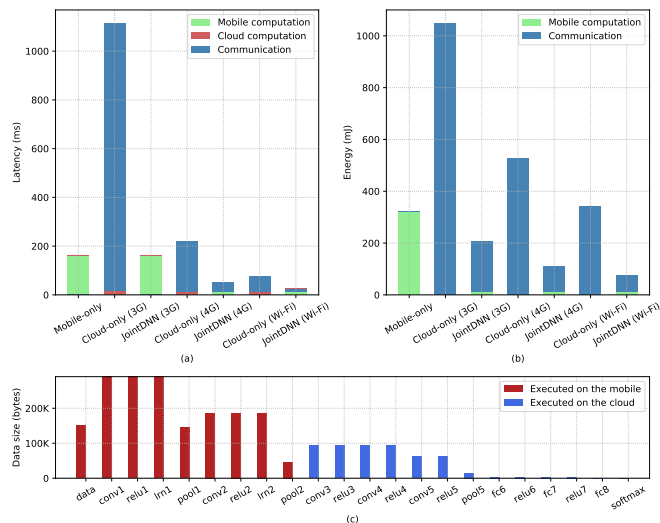


Fig. 12: (a) Execution time of AlexNet optimized for performance (b) Mobile energy consumption of AlexNet optimized for energy (c) Data size of the layers in AlexNet and the scheduled computation, where the first nine layers are computed on the mobile and the rest on the cloud, which is the optimal solution w.r.t. both performance and energy.

There are two general categories of compression techniques, lossy and loss-less [40]. In loss-less techniques, the exact original information is reconstructed. On the contrary, lossy techniques use approximations and the original data cannot be reconstructed. In our experiments, we examined the impact of compression of layer outputs using PNG, a loss-less technique, based on the encoding of frequent sequences in an image.

Even though the data type of DNN parameters in typical implementations is 32-bits floating-points, most image formats are based on 3-bytes RGB color triples. Therefore, to compress the layer in the same way as 2D pictures, the floating-point data should be quantized into 8-bits fixed-point. Recent studies show representing the parameters of DNNs with only 4-bits affects the accuracy, not more than

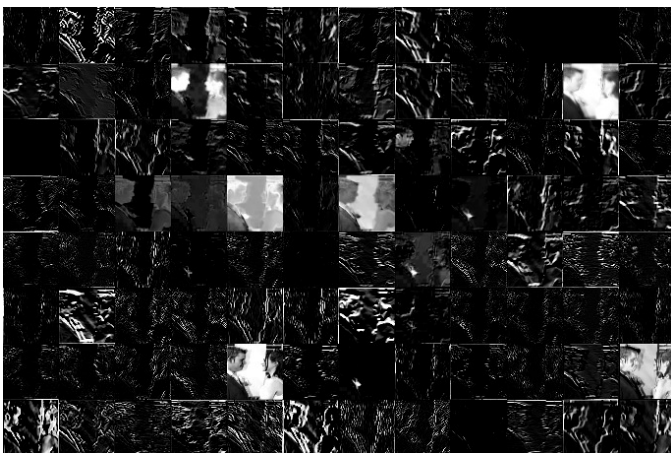


Fig. 13: Layer output after passing the input image through convolution, normalization and ReLU [39] layers. Channels are preserving the general structure of the input image and large ratio of the output data is black (zero) due to existence of *relu*. Tiling is used to put all 96 channels together.

1% [5]. In this work, we implemented our architectures with an 8-bits fixed-point and presented our baseline without any compression and quantization. The layers of CNN contain numerous channels of 2D matrices, each similar to an image. A simple method is to compress each channel separately. In addition to extra overhead of file header for each channel, this method will not take the best of the frequent sequence decoding of PNG. One alternative is locating different channels side by side, referred to as tiling, to form a large 2D matrix representing one layer as shown in Figure 13. It should be noted that 1D fully connected layers are very small and we did not apply compression on them.

The Compression Ratio (CR) is defined as the ratio of the size of the layer (8-bit) to the size of the compressed 2D matrix in PNG. Looking at the results of compression for two different CNN architectures in Figure 14, we can observe a high correlation between the ratio of pixels being zero (ZR) and CR. PNG can compress the layer data up to $5.8\times$ and $3.5\times$ by average, therefore the communication costs can be reduced drastically. By replacing the compressed layer’s output and adding the cost of the compression process itself, which is negligible compared to DNN operators, in JointDNN formulations, we achieve an extra $4.9\times$ and $4.6\times$ improvements in energy and latency on average, respectively.

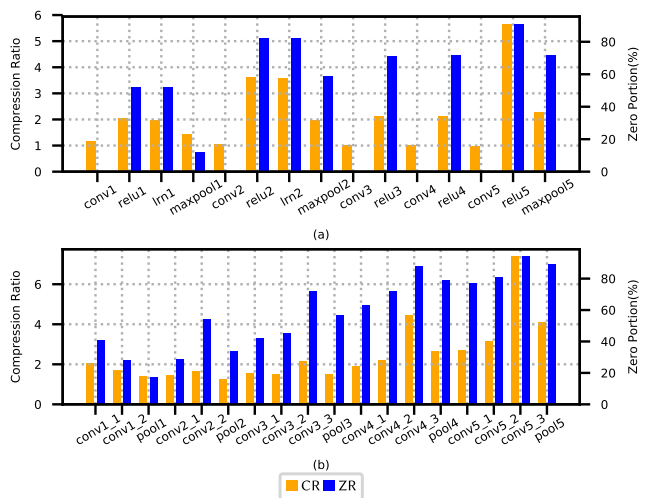


Fig. 14: Compression Ratio (CR) and ratio of zero valued neurons (ZR) for different layers of (a) AlexNet and (b) VGG16.

5 RELATED WORK AND COMPARISON

General Task Offloading Frameworks. There are existing prior arts focusing on offloading computation from the mobile to the cloud [15], [41], [42], [43], [44], [45], [46]. However, all these frameworks share a limiting feature that makes them impractical for computation partitioning of the DNN applications.

These frameworks are programmer annotations dependent as they make decisions about pre-specified functions, whereas JointDNN makes scheduling decisions based on the model topology and mobile network specifications in

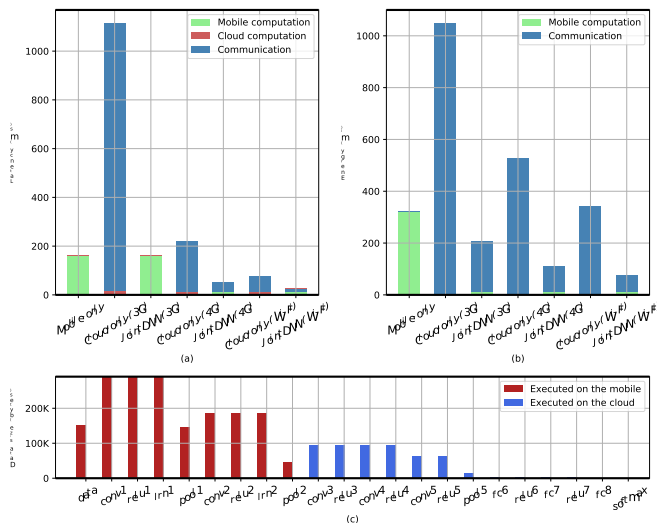


图12: (a) 针对性能优化的AlexNet的执行时间 (b) 针对能源优化的AlexNet的移动能源消耗 (c) AlexNet中各层的数据大小和计划的计算, 其中前九层在移动设备上计算, 其余在云上计算, 这是最佳解决方案。性能和能量。

压缩技术有两大类: 有损和无损[40]。在无损技术中, 准确的原始信息被重建。相反, 有损技术使用近似值并且无法重建原始数据。在我们的实验中, 我们检查了使用 PNG 压缩层输出的影响, PNG 是一种基于图像中频繁序列编码的无损技术。

尽管典型实现中 DNN 参数的数据类型是 32 位浮点, 但大多数图像格式都基于 3 字节 RGB 颜色三元组。因此, 为了以与 2D 图像相同的方式压缩该层, 浮点数据应量化为 8 位定点。最近的研究表明, 仅用 4 位表示 DNN 的参数会影响精度, 但不会超过

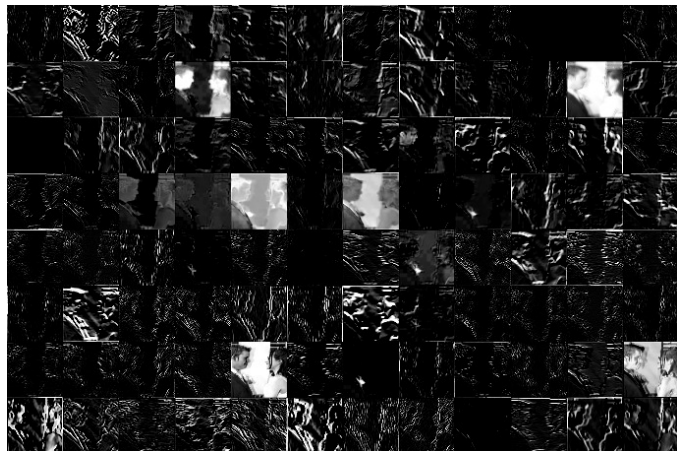


图 13: 输入图像经过卷积、归一化和 ReLU [39] 层后的层输出。通道保留了输入图像的一般结构, 并且由于 relu 的存在, 输出数据的大部分为黑色 (零)。平铺用于将所有 96 个通道组合在一起。

1% [5]。在这项工作中, 我们使用 8 位定点实现了我们的架构, 并在没有任何压缩和量化的情况下提出了我们的基线。CNN 的层包含大量二维矩阵通道, 每个通道都类似于图像。一种简单的方法是单独压缩每个通道。除了每个通道的文件头的额外开销之外, 该方法不会充分利用 PNG 的频繁序列解码。一种替代方法是并排放置不同的通道 (称为平铺), 以形成代表一层的大型 2D 矩阵, 如图 13 所示。应该注意的是, 1D 全连接层非常小, 我们没有对它们应用压缩。

压缩比 (CR) 定义为层的大小 (8 位) 与 PNG 中压缩的 2D 矩阵的大小之比。查看图 14 中两种不同 CNN 架构的压缩结果, 我们可以观察到为零 (ZR) 和 CR 的像素比率之间存在高度相关性。PNG 可以将图层数据平均压缩到 $5.8\times$ 和 $3.5\times$, 因此可以大大降低通信成本。通过替换压缩层的输出并添加压缩过程本身的成本 (与 DNN 算子相比, 在 JointDNN 公式中可以忽略不计), 我们在能量和延迟方面分别实现了额外的 $4.9\times$ 和 $4.6\times$ 改进。

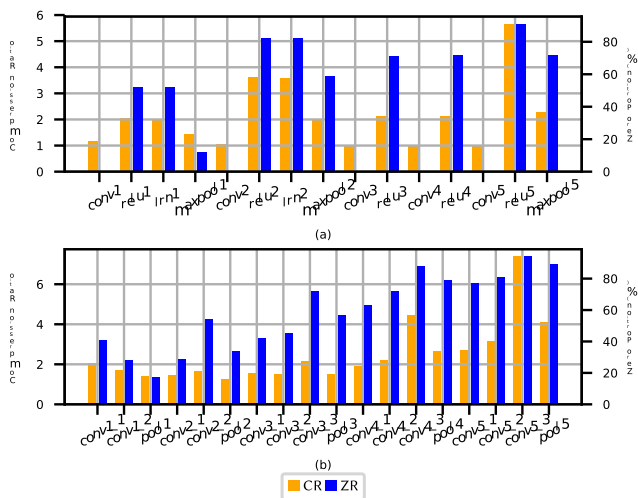


图 14: (a) AlexNet 和 (b) VGG16 不同层的压缩比 (CR) 和零值神经元 (ZR) 比率。

5 相关工作及比较

一般任务卸载框架。现有的现有技术专注于将计算从移动设备转移到云端[15]、[41]、[42]、[43]、[44]、[45]、[46]。然而, 所有这些框架都有一个限制特征, 这使得它们对于 DNN 应用程序的计算分区来说不切实际。

这些框架在对预先指定的功能做出决策时依赖于程序员注释, 而 JointDNN 则根据模型拓扑和移动网络规范做出调度决策。

run-time. Offloading in function level, cannot lead to efficient partition decisions due to layers of a given type within one architecture can have significantly different computation and data characteristics. For instance, a specific convolution layer structure can be computed on mobile or cloud in different models in the optimal solution.

Neurosurgeon [14] is the only prior art exploring a similar computation offloading idea in DNNs between the mobile device and the cloud server at layer granularity. Neurosurgeon assumes that there is only one data transfer point and the execution schedule of the efficient solution starts with mobile and then switches to the cloud, which performs the whole rest of the computations. Our results show this is not true especially for online training, where the optimal schedule of execution often follows the mobile-cloud-mobile pattern. Moreover, generative and autoencoder models follow a multi-transfer points pattern. Also, the execution schedule can start with the cloud especially in case of generative models where the input data size is large. Furthermore, inter-layer optimizations performed by DNN libraries are not considered in Neurosurgeon. Moreover, Neurosurgeon only schedules for optimal latency and energy, while JointDNN adapts to different scenarios including battery limitation, cloud server congestion, and QoS. Lastly, Neurosurgeon only targets simple CNN and ANN models, while JointDNN utilizes a graph-based approach to handle more complex DNN architectures like ResNet and RNNs.

6 ACKNOWLEDGEMENTS

This research was supported by grants from NSF SHF, DARPA MTO, and USC Annenberg Fellowship.

7 CONCLUSIONS AND FUTURE WORK

In this paper, we demonstrated that the status-quo approaches, cloud-only or mobile-only, are not optimal with regard to latency and energy. We reduced the problem of partitioning the computations in a DNN to shortest path problem in a graph. Adding constraints to the shortest path problem makes it NP-Complete, therefore, we also provided ILP formulations to cover different possible scenarios of limitations of mobile battery, cloud congestion, and QoS. The output data size in discriminative models is typically smaller than other layers in the network, therefore, last layers are expected to be computed on the cloud, while first layers are expected to be computed on the mobile. Reverse reasoning works for Generative models. Autoencoders have large input and output data sizes, which implies that the first and last layers are expected to be computed on the mobile. With these insights, the execution schedule of DNNs can possibly have various patterns depending on the model architecture in model cloud computing. JointDNN formulations are designed for feed-forward networks and its extension to recurrent neural networks will be studied as a future work.

REFERENCES

- [1] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M. P. Reyes, M.-L. Shyu, S.-C. Chen, and S. S. Iyengar, "A survey on deep learning: Algorithms, techniques, and applications," *ACM Comput. Surv.*, vol. 51, no. 5, pp. 92:1–92:36, Sep. 2018. [Online]. Available: <http://doi.acm.org/10.1145/3234150>
- [2] K.-S. Oh and K. Jung, "Gpu implementation of neural networks," vol. 37, pp. 1311–1314, 06 2004.
- [3] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le, M. Z. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, and A. Y. Ng, "Large scale distributed deep networks," in *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS'12. USA: Curran Associates Inc., 2012, pp. 1223–1231. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2999134.2999271>
- [4] M. S. Razlighi, M. Imani, F. Koushanfar, and T. Rosing, "Looknn: Neural network with no multiplication," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017, March 2017, pp. 1775–1780.
- [5] V. Sze, Y. Chen, T. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec 2017.
- [6] K. Skala, D. Davidovic, E. Afgan, I. Sovic, and Z. Sojat, "Scalable distributed computing hierarchy: Cloud, fog and dew computing," *Open Journal of Cloud Computing (OJCC)*, vol. 2, no. 1, pp. 16–24, 2015. [Online]. Available: <http://nbn-resolving.de/urn:nbn:de:101:1-201705194519>
- [7] A. Newsroom, "The future is here: iPhone X," <https://www.apple.com/newsroom/2017/09/the-future-is-here-iphone-x/>, 2017, [Online; accessed 15-January-2018].
- [8] H. Li, J. Yu, Y. Ye, and C. Bregler, "Realtime facial animation with on-the-fly correctives," *ACM Trans. Graph.*, vol. 32, no. 4, pp. 42:1–42:10, Jul. 2013.
- [9] A. E. Eshratifar, A. Esmaili, and M. Pedram, "Bottlenet: A deep learning architecture for intelligent mobile cloud computing services," in *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, July 2019, pp. 1–6.
- [10] —, "Towards collaborative intelligence friendly architectures for deep learning," in *20th International Symposium on Quality Electronic Design (ISQED)*, March 2019, pp. 14–19.
- [11] A. E. Eshratifar and M. Pedram, "Energy and performance efficient computation offloading for deep neural networks in a mobile cloud computing environment," in *Proceedings of the 2018 on Great Lakes Symposium on VLSI*, ser. GLSVLSI '18. New York, NY, USA: ACM, 2018, pp. 111–116. [Online]. Available: <http://doi.acm.org/10.1145/3194554.3194565>
- [12] Y. Pan, C.-A. Cheng, K. Saigol, K. Lee, X. Yan, E. Theodorou, and B. Boots, "Agile autonomous driving using end-to-end deep imitation learning," in *Robotics: Science and Systems*, 2018.
- [13] M. Nazemi, A. E. Eshratifar, and M. Pedram, "A hardware-friendly algorithm for scalable training and deployment of dimensionality reduction models on FPGA," in *Proceedings of the 19th IEEE International Symposium on Quality Electronic Design*, 2018.
- [14] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '17. New York, NY, USA: ACM, 2017, pp. 615–629.
- [15] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "Clonecloud: Elastic execution between mobile device and cloud," pp. 301–314, 2011.
- [16] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, pp. 328–339, 2017.
- [17] R. W. Ahmad, A. Gani, S. H. A. Hamid, F. Xia, and M. Shiraz, "A review on mobile application energy profiling: Taxonomy, state-of-the-art, and open research issues," *Journal of Network and Computer Applications*, vol. 58, pp. 42 – 59, 2015. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S1084804515002088>
- [18] S. Chetlur, C. Woolley, P. Vandermeresch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," *CoRR*, vol. abs/1410.0759, 2014. [Online]. Available: <http://arxiv.org/abs/1410.0759>
- [19] H. Qi, E. R. Sparks, and A. Talwalkar, "Paleo: A performance model for deep neural networks," in *ICLR*, 2017.
- [20] S. Hong and H. Kim, "An integrated gpu power and performance model," *SIGARCH Comput. Archit. News*, vol. 38, no. 3, pp. 280–289, Jun. 2010.
- [21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2015.
- [22] Z. Wang and J. Crowcroft, "Quality-of-service routing for supporting multimedia applications," *IEEE Journal on Selected Areas in Communications*, vol. 14, no. 7, pp. 1228–1234, Sep 1996.

运行时。由于一个架构内给定类型的层可能具有显著不同的计算和数据特征，因此功能级别的卸载无法导致有效的分区决策。例如，特定的卷积层结构可以在移动或云上以不同模型的最佳解决方案进行计算。

Neurosurgeon [14] 是唯一在层粒度上探索移动设备和云服务器之间的 DNN 中的类似负载计算思想的现有技术。神经外科医生假设只有一个数据传输点，并且高效解决方案的执行计划从移动设备开始，然后切换到云，由云执行其余的全部计算。我们的结果表明，情况并非如此，尤其是对于在线培训，在线培训的最佳执行计划通常遵循移动-云-移动模式。此外，生成和自动编码器模型遵循多传输点模式。此外，执行计划可以从云开始，特别是在输入数据量很大的生成模型的情况下。此外，Neurosurgeon 中不考虑 DNN 库执行的层间优化。此外，Neurosurgeon 仅针对最佳延迟和能量进行调度，而 JointDNN 则适应不同的场景，包括电池限制、云服务器拥塞和 QoS。最后，Neurosurgeon 仅针对简单的 CNN 和 ANN 模型，而 JointDNN 利用基于图的方法来处理更复杂的 DNN 架构，例如 ResNet 和 RNN。

6 致谢

这项研究得到了 NSF SHF、DARPA MTO 和 USC Annenberg Fellowship 的资助。

7 结论和未来工作

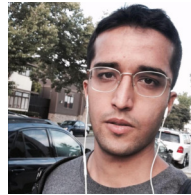
在本文中，我们证明了现状方法（仅云或仅移动）在延迟和能源方面并不是最佳的。我们将 DNN 中的计算分区问题简化为图中的最短路径问题。为最短路径问题添加约束使其成为 NP-Complete，因此，我们还提供了 ILP 公式来涵盖移动电池、云拥塞和 QoS 限制的不同可能场景。判别模型中的输出数据大小通常小于网络中的其他层，因此，最后一层预计在云端计算，而第一层预计在移动设备上计算。逆向推理适用于生成模型。自动编码器具有很大的输入和输出数据量，这意味着第一层和最后一层预计在移动设备上计算。有了这些见解，DNN 的执行计划可能会根据模型云计算中的模型架构而具有各种模式。JointDNN 公式是为前馈网络设计的，其对循环神经网络的扩展将作为未来的工作进行研究。

参考

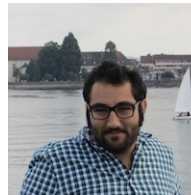
[1] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M. P. Reyes, M.-L. Schu, S.-C. Chen 和 S. S. Iyengar, “深度学习调查：算法、技术和应用”，*ACM Comput. Surv.*, 卷. 51, 没有. 5, 第 92:1–92:36, 2018 年 9 月。[在线]。可用：<http://doi.acm.org/10.1145/3234150>

[2] K.-S. Oh 和 K. Jung, “神经网络的 Gpu 实现”，卷. 37, 第 1311–1314 页, 2004 年 6 月。[3] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le, M. Z. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang 和 A. Y. Ng, “大规模分布式深度网络”，载于 *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, 系列. NIPS’12. 美国: Curran Associates Inc., 2012 年, 第 1223–1231 页。[在线的]。网址：<http://dl.acm.org/citation.cfm?id=2999134.2999271> [4] M. S. Razlighi, M. Imani, F. Koushanfar 和 T. Rosing, “Looknn: 无乘法的神经网络”，载于 *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017, 2017 年 3 月, 第 1775–1780 页。[5] V. Sze, Y. Chen, T. Yang 和 J. S. Emer, “深度神经网络的高效处理：教程和调查”，*Proceedings of the IEEE*, 卷. 105, 没有. 12, 第 2295–2329 页, 2017 年 12 月。[6] K. Skala, D. Davidovic, E. Afgan, I. Sovic 和 Z. Sojat, “可扩展的分布式计算层次结构：云、雾和露计算”，*Open Journal of Cloud Computing (OJCC)*, 卷. 2, 没有. 1, 第 16–24 页, 2015 年。[在线]。可用：<http://nbn-resolving.org/urn:nbn:de:101:1-201705194519> [7] A. Newsroom, “未来就在这里：iPhone X”，<https://www.apple.com/newsroom/2017/09/the-future-is-here-iphone-x/>, 2017 年, [在线; 2018 年 1 月 15 日访问]。[8] H. Li, J. Yu, Y. Ye 和 C. Bregler, “带有动态校正的实时面部动画”，*ACM Trans. Graph.*, 卷. 32, 没有. 4, 第 42:1–42:10, 2013 年 7 月。[9] A. E. Eshratifar, A. Esmaili 和 M. Pedram, “Bottlenet: 智能移动云计算服务的深度学习架构”，*2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2019 年 7 月, 第 1–6 页。[10] —, “面向深度学习的协作智能友好架构”，载于 *20th International Symposium on Quality Electronic Design (ISQED)*, 2019 年 3 月, 第 14–19 页。[11] A. E. Eshratifar 和 M. Pedram, “移动云计算环境中深度神经网络的能源和性能高效计算”，*Proceedings of the 2018 on Great Lakes Symposium on VLSI*, 系列. GLSVLSI’18. 美国纽约州纽约: ACM, 2018 年, 第 111–116 页。[在线的]。可用：<http://doi.acm.org/10.1145/3194554.3194565> [12] Y. Pan, C.-A. Cheng, K. Saigol, K. Lee, X. Yan, E. Theodorou 和 B. Boots, “使用端到端深度模仿学习的敏捷自动驾驶”，*Robotics: Science and Systems*, 2018 年。[13] M. Nazemi, A. E. Eshratifar 和 M. Pedram, “一种硬件友好的算法，用于在 FPGA 上进行可扩展训练和部署降维模型”，*Proceedings of the 19th IEEE International Symposium on Quality Electronic Design*, 2018。[14] Y. Kang, J. Hauswald, C. Gau, A. Rovinski, T. Mudge, J. Mars 和 L. Tang, “神经外科医生：云和移动边缘之间的协作智能”，载于 *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 序列号. ASPLOS’17. 美国纽约州纽约: ACM, 2017 年, 第 615–629 页。[15] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik 和 A. Patti, “Clonecloud: 移动设备和云之间的弹性执行”，第 301–314 页, 2011 年。[16] S. Teerapittayanon, B. McDanel 和 H. T. Kung, “云端、边缘和终端设备上的分布式深度神经网络”，*2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, 第 301 页. 328–339, 2017。[17] R. W. Ahmad, A. Gani, S. H. A. Hamid, F. Xia 和 M. Shiraz, “移动应用能源分析综述：分类学、最新技术和开放研究问题”，*Journal of Network and Computer Applications*, 卷. 58, 第 42–59 页, 2015 年。[在线]。网址：<http://www.sciencedirect.com/science/article/pii/S1084804515002088> [18] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro 和 E. Shelhamer, “cudnn: 深度学习的高效原语”，*CoRR*, 卷. 绝对/1410.0759, 2014。[在线]。可用：<http://arxiv.org/abs/1410.0759> [19] H. Qi, E. R. Sparks 和 A. Talwalkar, “Paleo: 深度神经网络的性能模型”，*ICLR*, 2017 年。[20] S. Hong 和 H. Kim, “集成 GPU 功率和性能模型”，*SIGARCH Comput. Archit. News*, 卷. 38, 没有. 3, 第 280–289 页, 2010 年 6 月。[21] K. He, X. Zhang, S. Ren 和 J. Sun, “用于图像识别的深度残差学习”，*2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 第 770–778 页, 2015 年。[22] Z. Wang 和 J. Crowcroft, “支持多媒体应用的服务质量路由”，*IEEE Journal on Selected Areas in Communications*, 卷. 14, 没有. 7, 第 1228–1234 页, 1996 年 9 月。

- [23] A. Juttner, B. Szviatovski, I. Mecs, and Z. Rajko, "Lagrange relaxation based method for the qos routing problem," vol. 2, pp. 859–868 vol.2, 2001.
- [24] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems* 25, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 1097–1105.
- [25] Y. Bengio and Y. LeCun, Eds., *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014. [Online]. Available: <https://openreview.net/group?id=ICLR.cc/2014>
- [26] —, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. [Online]. Available: <https://iclr.cc/archive/www/doku.php%3Fid=iclr2015:accepted-main.html>
- [27] A. Y. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, S. Sengupta, A. Coates, and A. Y. Ng, "Deep speech: Scaling up end-to-end speech recognition," *CoRR*, vol. abs/1412.5567, 2014. [Online]. Available: <http://arxiv.org/abs/1412.5567>
- [28] Y. Bengio and Y. LeCun, Eds., *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014. [Online]. Available: <https://openreview.net/group?id=ICLR.cc/2014>
- [29] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems* 27, Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2014, pp. 2672–2680. [Online]. Available: <http://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf>
- [30] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2786–2793, 2016.
- [31] *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*. IEEE Computer Society, 2015. [Online]. Available: <https://ieeexplore.ieee.org/xpl/conhome/7293313/proceeding>
- [32] *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 2017. [Online]. Available: <https://ieeexplore.ieee.org/xpl/conhome/8097368/proceeding>
- [33] N. Corporation, "Jetson TX2 Module," <https://developer.nvidia.com/embedded/buy/jetson-tx2>, 2018, [Online; accessed 15-January-2018].
- [34] T. I. Incorporated, "INA Current/Power Monitor," <http://www.ti.com/product/INA226>, 2018, [Online; accessed 15-January-2018].
- [35] N. Corporation, "TESLA DATA CENTER GPUS FOR SERVERS," <http://www.nvidia.com/object/tesla-servers.html>, 2018, [Online; accessed 15-January-2018].
- [36] OpenSignal.com, "State of Mobile Networks: USA," <https://opensignal.com/reports/2017/08/usa/state-of-the-mobile-network>, 2017, [Online; accessed 15-January-2018].
- [37] —, "United States Speedtest Market Report," <http://www.speedtest.net/reports/united-states/>, 2017, [Online; accessed 15-January-2018].
- [38] J. Huang, F. Qian, A. Gerber, Z. M. Mao, S. Sen, and O. Spatscheck, "A close examination of performance and power characteristics of 4g lte networks," in *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys '12. New York, NY, USA: ACM, 2012, pp. 225–238.
- [39] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, G. Gordon, D. Dunson, and M. Dudk, Eds., vol. 15. Fort Lauderdale, FL, USA: PMLR, 11–13 Apr 2011, pp. 315–323. [Online]. Available: <http://proceedings.mlr.press/v15/glorot11a.html>
- [40] T. M. Cover and J. A. Thomas, *Elements of Information Theory (Wiley Series in Telecommunications and Signal Processing)*. Wiley-Interscience, 2006.
- [41] M.-R. Ra, A. Sheth, L. Mummert, P. Pillai, D. Wetherall, and R. Govindan, "Odessa: Enabling interactive perception applications on mobile devices," pp. 43–56, 2011. [Online]. Available: <http://doi.acm.org/10.1145/1999995.2000000>
- [42] M. S. Gordon, D. A. Jamshidi, S. Mahlke, Z. M. Mao, and X. Chen, "Comet: Code offload by migrating execution transparently," pp. 93–106, 2012. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2387880.2387890>
- [43] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl, "Maui: Making smartphones last longer with code offload," pp. 49–62, 2010. [Online]. Available: <http://doi.acm.org/10.1145/1814433.1814441>
- [44] X. Wang, X. Liu, Y. Zhang, and G. Huang, "Migration and execution of javascript applications between mobile devices and cloud," pp. 83–84, 2012.
- [45] Y. Zhang, G. Huang, X. Liu, W. Zhang, H. Mei, and S. Yang, "Refactoring android java code for on-demand computation offloading," *SIGPLAN Not.*, vol. 47, no. 10, pp. 233–248, Oct. 2012.
- [46] K. Kumar, J. Liu, Y.-H. Lu, and B. Bhargava, "A survey of computation offloading for mobile systems," *Mobile Networks and Applications*, vol. 18, no. 1, pp. 129–140, Feb 2013. [Online]. Available: <https://doi.org/10.1007/s11036-012-0368-0>



Amir Erfan Eshratifar Amir Erfan Eshratifar received dual B.S. degrees in Electrical Engineering and Computer Science from Sharif University of Technology, Tehran, Iran in 2017. He is currently a Ph.D. student in Ming Hsieh Department of Electrical Engineering, University of Southern California (USC), Los Angeles, CA, USA since 2017.



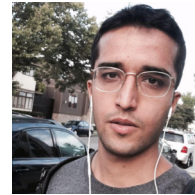
Mohammad Saeed Abrishami Mohammad Saeed Abrishami received his B.S. degree in Electrical Engineering from University of Tehran, Tehran, Iran in 2014. He is currently a Ph.D. student in Ming Hsieh Department of Electrical Engineering, University of Southern California (USC), Los Angeles, CA, USA since 2014.



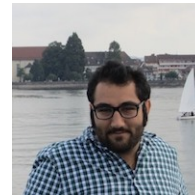
Massoud Pedram Massoud Pedram (F01) received the B.S. degree in electrical engineering from the California Institute of Technology, Pasadena, CA, USA, in 1986, and the M.S. and Ph.D. degrees in electrical engineering and computer sciences from the University of California Berkeley, CA, USA, in 1989 and 1991, respectively. In 1991, he joined the Ming Hsieh Department of Electrical Engineering, University of Southern California (USC), Los Angeles, CA, USA, where he is currently the Charles Lee Powell Professor of USC Viterbi School of Engineering.

[23] A. Juttner, B. Szviatovski, I. Mecs 和 Z. Rajko, “基于拉格朗日松弛的 qos 路由问题方法”, 卷. 2, 第 859-868 页, 第 2 卷, 2001 年。[24] A. Krizhevsky, I. Sutskever 和 G. E. Hinton, “深度卷积神经网络的 I magenet 分类”, 见 *Advances in Neural Information Processing Systems* 25, F. Pereira, C. J. C. Burges, L. Bottou 和 K. Q. Weinberger, 编辑。Curran Associates, Inc., 2012 年, 第 1097–1105 页。[25] Y. Bengio 和 Y. LeCun, 编辑, *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014 年。[在线]。可用: <https://openreview.net/group?id=ICLR.cc/2014> [26] —, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015。[在线]。可用: <https://iclr.cc/archive/www/doku.php%3Fid=iclr2015:accepted-main.html> [27] A. Y. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, S. Sengupta, A. Coates 和 A. Y. Ng, “Deep” 语音: 扩展端到端语音识别”, *CoRR*, 卷. 绝对/1412.5567, 2014。[在线]。可用: <http://arxiv.org/abs/1412.5567> [28] Y. Bengio 和 Y. LeCun, 编辑, *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014 年。[在线]。可用: <https://openreview.net/group?id=ICLR.cc/2014> [29] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville 和 Y. Bengio, “生成对抗网络”, *Advances in Neural Information Processing Systems* 27, Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence 和 K. Q. Weinberger, 编辑。Curran Associates, Inc., 2014 年, 第 2672–2680 页。[在线的]。网址: <http://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf> [30] C. Finn 和 S. Levine, “用于规划机器人运动的深度视觉远见”, *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 第 2786–2793 页, 2016 年。[31] *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*。IEEE 计算机协会, 2015 年。[在线]。可用: <https://ieeexplore.ieee.org/xpl/conhome/7293313/proceeding> [32] *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*。IEEE 计算机协会, 2017 年。[在线]。可用: <https://ieeexplore.ieee.org/xpl/conhome/8097368/proceeding> [33] N. Corporation, “Jetson TX2 模块”, <https://developer.nvidia.com/embedded/buy/jetson-tx2>, 2018, [在线; 2018 年 1 月 15 日访问]。[34] T. I. Incorporated, “INA 电流/功率监视器”, <http://www.ti.com/product/INA226>, 2018, [在线; 2018 年 1 月 15 日访问]。[35] N. Corporation, “用于服务器的特斯拉数据中心 GPU”, <http://www.nvidia.com/object/tesla-servers.html>, 2018 年, [在线; 2018 年 1 月 15 日访问]。[36] OpenSignal.com, “移动网络状况: 美国”, <https://opensignal.com/reports/2017/08/usa/state-of-the-mobile-network>, 2017 年, [在线; 2018 年 1 月 15 日访问]。[37] —, 《美国测速市场报告》, <http://www.speedtest.net/reports/united-states/>, 2017 年, [在线; 2018 年 1 月 15 日访问]。[38] J. Huang, F. Qian, A. Gerber, Z. M. Mao, S. Sen 和 O. Spatscheck, “对 4g lte 网络性能和功率特性的仔细检查”, 见 *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services*, 系列. MobiSys '12。美国纽约州纽约: ACM, 2012 年, 第 225–238 页。[39] X. Glorot, A. Borde 和 Y. Bengio, “深度稀疏整流神经网络”, 载于 *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 系列. 机器学习研究论文集, G. Gordon, D. Dunson 和 M. Dudk, 编辑, 卷. 15。美国佛罗里达州劳德代尔堡: PMLR, 2011 年 4 月 11-13 日, 第 315-323 页。[在线的]。可用: <http://proceedings.mlr.press/v15/glorot11a.html> [40] T. M. Cover 和 J. A. Thomas, *Elements of Information Theory* (Wiley Series in Telecommunications and Signal Processing)。Wiley-Interscience, 2006。[41] M.-R. Ra, A. Sheth, L. Mummert, P. Pillai, D. Wetherall 和 R. Govindan, “散德萨: 在移动设备上启用交互式感知应用程序”, 第 43-56 页, 2011 年。[在线]。网址: <http://doi.acm.org/10.1145/1999995.2000000> [42] M. S. Gordon, D. A. Jamshidi, S. Mahlke, Z. M. Mao 和 X. Chen, “Comet: 通过迁移执行来卸载代码

透明地”, 第 93–106 页, 2012 年。[在线]。可用: <http://dl.acm.org/itition.cfm?id=2387880.2387890> [43] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra 和 P. Bahl, “毛伊岛: 通过代码卸载让智能手机的使用寿命更长”, 第 49-62 页, 2010 年。[在线]。可获取: <http://doi.acm.org/10.1145/1814433.1814441> [44] X. Wang, X. Liu, Y. Zhang 和 G. Huang, “移动设备和云之间 javascript 应用程序的迁移和执行”, 第 10 页。83-84, 2012。[45] Y. Zhang, G. Huang, X. Liu, W. Zhang, H. Mei 和 S. Yang, “重构 android java 代码以实现按需计算溢出”, 第 47 卷, 第 10 期, 第 233-248 页, 2012 年 10 月。[46] K. Kumar, J. Liu, Y.-H. Lu 和 B. Bhargava, “移动系统的计算概述”, 第 18 卷, 第 129–140 页, 2013 年 2 月。[在线]。



Amir Erfan Eshratifar Amir Erfan Eshratifar 获得了双学士学位。他于 2017 年在伊朗德黑兰谢里夫理工大学获得电气工程和计算机科学博士学位。自 2017 年起, 谢明在美国加利福尼亚州洛杉矶南加州大学 (USC) 电气工程系就读。



穆罕默德·赛义德·阿布里沙米 (Mohammad Saeed Abrishami) 穆罕默德·赛义德·阿布里沙米 (Mohammad Saeed Abrishami) 获得理学学士学位 2014 年获得伊朗德黑兰德黑兰大学电气工程学士学位。目前正在攻读博士学位。自 2014 年起, 谢明在美国加利福尼亚州洛杉矶南加州大学 (USC) 电气工程系就读。



Massoud Pedram Massoud Pedram (F01) 获得了理学学士学位 1986 年获得美国加利福尼亚州帕萨迪纳市加州理工学院电气工程学士学位, 和博士学位。分别于 1989 年和 1991 年获得美国加州大学伯克利分校电气工程和计算机科学学位。1991 年, 他加入美国加利福尼亚州洛杉矶南加州大学 (USC) 谢明谢电气工程系, 现任该系的 Charles Lee

南加州大学维特比工程学院鲍威尔教授。